

SYNTH

A virtual-analog polysynth — source code deposit

Copyright © 2026 John L Farmer

Licensed under the GNU Affero General Public License v3.0

US Copyright Office registration deposit — Literary Work (computer program).

Author / claimant: John L Farmer

Prepared: 2026-08-15

Corpus: first-party product source (Source/), 75 files, 21013 source lines.

Excludes: tests, build tooling, and all third-party code (JUICE and vendored libraries).

Full listing paginates to 288 pages at 76 lines/page.

Per USCO practice for programs over 50 pages, this deposit contains the FIRST 25 and LAST 25 pages.

The omitted middle pages (if any) are marked by a gap in the page numbering.

See [copyright-deposit-manifest.md](#) for the exact file-to-page mapping.

===== FILE 1/75: Source/AppInfo.h (84 lines) =====

// Synth — Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.

#pragma once

#include <juce_core/juce_core.h>

```
// =====
// Single source of truth for the app's user-facing identity and its on-disk
// config location, plus a one-time migration from the pre-rename "VASynth"
// locations to the new "synth" ones so an existing rig keeps its presets, MIDI
// profiles, and (standalone) audio/MIDI settings after the rename.
//
// The plugin's display name (VST3 filename, window title) comes from
// PRODUCT_NAME in CMake; keep it in sync with kName here. Config subdirectories
// are chosen explicitly (not derived from the plugin name) so they are stable
// and testable across the plugin and standalone.
// =====
```

namespace AppInfo

```
{
    // Display identity (UI / About / banner). kDisplayName is intentionally SEPARATE from kName:
    inline constexpr const char* kDisplayName = "Synth";    // product name shown to the user
    inline constexpr const char* kVendor      = "Farmer";    // author / vendor

    // On-disk config KEY — the config-subdirectory name. Kept lowercase and STABLE (not the display
    // name) so existing installs' presets / MIDI profiles / settings never move when the display name
    // changes. Do not repoint this without a migration.
    inline constexpr const char* kName       = "synth";      // config-dir key (NOT the display name)
    inline constexpr const char* kLegacyName = "VASynth";     // pre-rename config dir

    // Base user-data directory (Linux: ~/.config, Windows: AppData/Roaming, etc.).
    inline juce::File dataRoot()
    {
        return juce::File::getSpecialLocation (juce::File::userApplicationDataDirectory);
    }

    inline juce::File configDir()      { return dataRoot().getChildFile (kName); }
    inline juce::File legacyConfigDir() { return dataRoot().getChildFile (kLegacyName); }

    inline juce::File presetDir()      { auto d = configDir().getChildFile ("presets");    d.createDirectory();
    return d; }
    inline juce::File midiProfileDir() { auto d = configDir().getChildFile ("midi-profiles"); d.createDirectory();
    return d; }
    inline juce::File multiDir()       { auto d = configDir().getChildFile ("multis");      d.createDirectory();
    return d; }
    inline juce::File kitDir()          { auto d = configDir().getChildFile ("kits");       d.createDirectory();
    return d; }
    inline juce::File samplesDir()      { auto d = configDir().getChildFile ("samples");    d.createDirectory();
    return d; } // I2: managed sample library (<md5>.<ext>)
    inline juce::File logFile()         { return configDir().getChildFile (juce::String (kName) + ".log"); }

    // Core migration (pure, so it is unit-testable with temp dirs): copy each of
    // the config SUBDIRS from `legacy` to `dest`, but only when the destination
    // sub-dir does not yet exist. Per-subdir + existence-gated makes it idempotent
    // and not blocked by unrelated files (e.g. a freshly-created log) already in
    // `dest`. Returns the number of subdirs actually copied.
    inline int migrateConfigTree (const juce::File& legacy, const juce::File& dest)
    {
        if (! legacy.isDirectory()) return 0;
        dest.createDirectory();
        int copied = 0;
        for (const char* sub : { "presets", "midi-profiles" })
        {
            auto from = legacy.getChildFile (sub);
            auto to   = dest.getChildFile (sub);
            if (from.isDirectory() && ! to.exists())
            {
                to.createDirectory();
                if (from.copyDirectoryTo (to)) ++copied;    // copies the user's saved files
            }
        }
        return copied;
    }
}

// One-time migration of the real legacy config. Safe on every launch: a no-op
// once migrated or when there is no legacy data (fresh machine / CI).
inline void migrateLegacyConfig() { migrateConfigTree (legacyConfigDir(), configDir()); }
```

```

// Standalone-only: migrate the JUCE PropertiesFile (audio/MIDI settings) from
// the old "<legacy>.settings" to "<name>.settings" in the same folder, once.
// `settingsFolder` is where the standalone stores its .settings file.
inline void migrateLegacySettings (const juce::File& settingsFolder)
{
    auto to = settingsFolder.getChildFile (juce::String (kName) + ".settings");
    if (to.existsAsFile()) return;
    // Pre-rename the standalone used the product name "VA Synth".
    auto from = settingsFolder.getChildFile ("VA Synth.settings");
    if (from.existsAsFile())
        from.copyFileTo (to);
}
}

===== FILE 2/75: Source/BuildStamp.cpp (27 lines) =====
// Synth – Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.
// Emits a uniquely-greppable build stamp into every VASynth binary (VST3 + Standalone) so the
// gate (run-all-checks.sh) can PROVE the artefact on disk was built from the gated commit.
// "Gate green" then means "the binaries ARE the gated code", not merely "an artefact exists".
//
// The stamp comes from VersionInfo.h, which cmake/gen_version.cmake regenerates on every build
// with the currently-built commit hash (+ a "+" suffix for an uncommitted tree). The symbol is
// otherwise unreferenced, so __attribute__((used)) keeps it past the linker's --gc-sections.
#include "VersionInfo.h"

// "used" stops the COMPILER discarding it; "retain" (GCC 11+/Clang, SHF_GNU_RETAIN) stops the
// LINKER's --gc-sections collecting the section. It is also referenced from PluginProcessor.cpp,
// so even a toolchain without "retain" keeps it – belt and suspenders.
#if defined(__has_attribute)
    #if __has_attribute(retain)
        #define VASYNTH_KEEP __attribute__((used, retain))
    #endif
#else
    #define VASYNTH_KEEP
#endif
#ifdef VASYNTH_KEEP
    #if defined(__GNUC__) || defined(__clang__)
        #define VASYNTH_KEEP __attribute__((used))
    #else
        #define VASYNTH_KEEP // MSVC keeps extern non-static data by default
    #endif
#endif

extern "C" VASYNTH_KEEP const char vasyntb_build_stamp[] = VASYNTH_BUILD_STAMP;

===== FILE 3/75: Source/DSP/ADSREnvelope.h (133 lines) =====
// Synth – Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.
#pragma once
#include <cmath>
#include <algorithm>

// =====
// ADSR envelope, hand-rolled, with one-pole exponential segments.
//
// WHY EXPONENTIAL:
// Analog envelope generators charge/discharge capacitors, producing
// exponential curves. Linear digital envelopes sound flat and "computery" –
// the attack in particular feels sluggish and the release feels abrupt.
// We model each segment as a one-pole toward an (overshooting) target,
// which is both cheap and exactly what an RC circuit does.
//
// The attack targets slightly *above* 1.0 and we clip at 1.0 – this is the
// standard trick to make exponential attacks reach the top in finite time
// (a pure exponential never actually arrives).
// =====

class ADSREnvelope
{
public:
    enum class Stage { Idle, Attack, Decay, Sustain, Release };

    void prepare (double newSampleRate)
    {
        sampleRate = newSampleRate;
        reset();
    }
}

```

```

void reset()
{
    stage = Stage::Idle;
    level = 0.0;
    choking = false;
}

void setParameters (double attackS, double decayS, double sustainLvl, double releaseS)
{
    attackCoef = coefForTime (attackS);
    decayCoef  = coefForTime (decayS);
    releaseCoef = coefForTime (releaseS);
    sustain    = std::clamp (sustainLvl, 0.0, 1.0);
}

void noteOn()
{
    // Retrigger from current level, not zero -> no click on fast retrigs.
    stage = Stage::Attack;
    choking = false;           // a retrigger cancels any in-progress choke
}

void noteOff()
{
    if (stage != Stage::Idle)
        stage = Stage::Release;
}

// Hard steal: fast fade used by the voice allocator to avoid clicks.
// ~4 ms – long enough to be click-free, short enough that steal() completes
// well under the 10 ms budget even from a full-level note.
void quickRelease()
{
    chokeCoef = coefForTime (0.004);
    choking    = true;        // survives the per-render setParameters()
    stage      = Stage::Release;
}

bool isActive() const { return stage != Stage::Idle; }

// Key-up: the note has been released and is fading (or being quick-released/choked). Used by
// voice stealing to prefer a fading voice over a still-held one (#141).
bool inRelease() const { return stage == Stage::Release; }

// Current level WITHOUT advancing – for control-rate modulation targets (e.g.
// filter-envelope -> pitch) sampled once per render chunk.
float getLevel() const { return static_cast<float> (level); }

float nextSample()
{
    switch (stage)
    {
        case Stage::Attack:
            level += attackCoef * (attackTarget - level);
            if (level >= 1.0) { level = 1.0; stage = Stage::Decay; }
            break;

        case Stage::Decay:
            level += decayCoef * (sustain - level);
            if (level <= sustain + 1.0e-4) { level = sustain; stage = Stage::Sustain; }
            break;

        case Stage::Sustain:
            level = sustain;
            break;

        case Stage::Release:
            level += (choking ? chokeCoef : releaseCoef) * (0.0 - level);
            if (level <= 1.0e-5) { level = 0.0; stage = Stage::Idle; choking = false; }
            break;

        case Stage::Idle:
            level = 0.0;
            break;
    }
    return static_cast<float> (level);
}

```

```

}

private:
    // One-pole coefficient calibrated so a segment converges to within ~-60 dB
    // (0.001) of its target span in `seconds`. Derivation: after n = seconds*sr
    // samples the remaining fraction (1-coef)^n must equal 1e-3, so
    // coef = 1 - exp(ln(1e-3) / (seconds*sr)) = 1 - exp(-6.908 / (seconds*sr)).
    // This makes nominal envelope times honest: a 0.2 s release reaches -80 dB
    // in ~1.3x the release time, and quickRelease settles in a few ms.
    double coefForTime (double seconds) const
    {
        seconds = std::max (seconds, 0.0005);
        return 1.0 - std::exp (kConvergence / (seconds * sampleRate));
    }

    static constexpr double kConvergence = -6.907755278982137; // ln(1e-3)

    static constexpr double attackTarget = 1.3; // overshoot -> finite attack

    double sampleRate = 44100.0;
    double level = 0.0, sustain = 0.8;
    double attackCoef = 0.0, decayCoef = 0.0, releaseCoef = 0.0;
    double chokeCoef = 0.0; // fast release for steal/choke, immune to setParameters
    bool choking = false;
    Stage stage = Stage::Idle;
};

```

==== FILE 4/75: Source/DSP/Arpeggiator.h (252 lines) =====

// Synth — Copyright © 2026 John L Farmer. Licensed under AGPLV3; see LICENSE.

#pragma once

#include <array>

#include <algorithm>

#include <cstdint>

```

// =====
// Arpeggiator + 16-step gate — JUCE-free, RT-safe (fixed storage, no alloc/lock).
//
// Held notes form a chord; the arp steps through them on an internal clock in the
// chosen order (Up / Down / Up-Down / Random / As-played), spread over `octaves`,
// with a per-step velocity pattern (0 = rest). GATE sets note length, SWING delays
// the off-beat 16ths, LATCH/HOLD sustain the chord after the keys are released.
//
// The owner (the processor) feeds held-note edges via noteOn/noteOff (block-granular
// is fine — the arp re-times output to its clock) and calls process() once per block
// to emit clock-accurate note events (sample offsets) through a callback.
// =====

```

```

class Arpeggiator
{
public:
    enum Mode { Up = 0, Down, UpDown, Random, AsPlayed };
    static constexpr int kMaxHeld = 16;
    static constexpr int kNumSteps = 16;

    struct Config
    {
        bool enabled = false;
        bool latch = false; // sustain the chord after release (LATCH or HOLD)
        int mode = Up;
        int octaves = 1; // 1..4
        float gate = 0.5f; // 0..1 of a step
        float swing = 0.0f; // 0..~0.7 (delays odd 16ths)
        double samplesPerStep = 6000; // 16th-note length in samples (from tempo)
        // Per-step ABSOLUTE velocity (#136): 0 = rest (step off), else the step's velocity
        // (0.1..2.0 = 10..200 %, where 100 % = full MIDI). When the ARP is active this takes
        // PRECEDENCE over how hard the note was played — the step value IS the output velocity,
        // so a pattern sounds identical however you touch the keys. > 100 % accents via the
        // voice's velocity > 1.0 boost. The velocity belongs to the STEP, not the note.
        std::array<float, kNumSteps> steps { };
    };

    void setConfig (const Config& c) { cfg = c; }
    bool enabled() const { return cfg.enabled; }
    int currentStep() const { return stepIndex; } // -1 = idle; for the UI playhead

    void reset()

```

```

{
    heldCount = keysDown = 0;
    activeNote = -1;
    stepIndex = -1;
    sampleInStep = 0.0;
    seqCursor = 0;
    started = false;
}

// A held-note edge (from the played surface). Block-granular timing is fine.
void noteOn (int note, float vel)
{
    if (cfg.latch && keysDown == 0) heldCount = 0;    // new physical chord starts fresh
    ++keysDown;
    addHeld (note, vel);
}
void noteOff (int note)
{
    if (keysDown > 0) --keysDown;
    if (! cfg.latch) removeHeld (note);
}

// Re-lock the step PHASE to the shared grid (task #53) WITHOUT resetting the pattern index.
// The owner calls this at each bar boundary: it forces the NEXT step to fire on the downbeat
// (setting sampleInStep to a full step so process() emits at offset 0), so the arp's steps stay
// on the beat grid with bounded drift. But it advances the pattern normally (stepIndex+1) – it
// does NOT snap back to step 0 at the measure. So striking a key mid-bar starts the arp there and
// it free-runs its pattern locked to the grid, instead of skipping to the start each bar.
void realign() { sampleInStep = stepLength(); }

// #145 (guard 3): PHASE-ONLY bar-boundary re-anchor, same latent double-fire class as the step
// sequencer. realign() forces the next step to fire on the downbeat (sampleInStep = a full
// step), which advances the pattern an EXTRA step whenever the natural clock already crossed the
// boundary in the prior block. This corrects only the fractional phase when the arp and transport
// agree on the current step – no forced (extra) step.
void realignPhase (int transportStep, double transportInto)
{
    if (! started) return;
    const int g = ((transportStep % kNumSteps) + kNumSteps) % kNumSteps;
    // The arp's stepIndex tracks the transport grid (it is started via startAtGrid). If it has
    // ALREADY advanced onto grid step g (a mid-block boundary: the natural clock fired it in the
    // prior block), only snap the phase – do NOT force, or the pattern double-advances (the old
    // realign's latent bug). Otherwise grid step g is due: fire the next pattern step once, on the
    // downbeat (stepIndex is untouched, so the pattern advances normally – no reset).
    if (stepIndex == g)
        sampleInStep = std::clamp (transportInto, 0.0, stepLength());
    else
        sampleInStep = stepLength();
}

// Start the arp ALREADY LOCKED to the shared grid instead of firing step 0 the instant it's
// enabled: `g` = the bar's current 16th index, `intoStep` = samples into it. So turning the arp
// on mid-bar picks up on the beat the transport is at, matching the sequencer (task #127). A
// strike lands within block tolerance; a boundary hit still fires this step.
void startAtGrid (int g, double intoStep)
{
    started = true;
    g = ((g % kNumSteps) + kNumSteps) % kNumSteps;
    const double stepLen = cfg.samplesPerStep;
    if (intoStep < 1.0)
    {
        stepIndex = (g - 1 + kNumSteps) % kNumSteps;
        sampleInStep = stepLen;
    }
    else
    {
        stepIndex = g;
        sampleInStep = std::min (intoStep, stepLen);
    }
}

// Emit clock-accurate events for this block. emit(sampleOffset, note, velocity, isOn).
template <typename Emit>
void process (int numSamples, Emit&& emit)
{
    if (! cfg.enabled) return;

```

```

    if (heldCount == 0) // nothing held -> stop + rest the clock
    {
        if (activeNote >= 0) { emit (0, activeNote, 0.0f, false); activeNote = -1; }
        stepIndex = -1; sampleInStep = 0.0; seqCursor = 0; started = false;
        return;
    }

    double pos = 0.0;
    if (! started) { started = true; stepIndex = -1; sampleInStep = 0.0; doStep (0, emit); }

    const double eps = 1.0e-6;
    while (pos < numSamples)
    {
        const double stepLen = stepLength();
        const double toStep = stepLen - sampleInStep;
        const double toGate = (activeNote >= 0) ? gateRemaining : 1.0e18;
        const double adv = std::min ({ toStep, toGate, (double) numSamples - pos });

        pos += adv; sampleInStep += adv;
        if (activeNote >= 0) gateRemaining -= adv;

        bool acted = false;
        if (activeNote >= 0 && gateRemaining <= eps)
        { emit ((int) pos, activeNote, 0.0f, false); activeNote = -1; acted = true; }

        if (sampleInStep >= stepLen - eps)
        { sampleInStep -= stepLen; doStep ((int) pos, emit); acted = true; }

        if (! acted) break; // consumed the rest of the block
    }
}

private:
double stepLength() const
{
    const double s = (double) cfg.swing; // odd 16ths shortened, evens lengthened
    return cfg.samplesPerStep * ((stepIndex % 2 == 0) ? (1.0 + s) : (1.0 - s));
}

template <typename Emit>
void doStep (int pos, Emit&& emit)
{
    if (activeNote >= 0) { emit (pos, activeNote, 0.0f, false); activeNote = -1; }
    stepIndex = (stepIndex + 1) % kNumSteps;
    const float sv = cfg.steps[(std::size_t) stepIndex];
    if (heldCount > 0 && sv > 1.0e-3f)
    {
        int note; float playedVel;
        pickNote (note, playedVel);
        (void) playedVel; // intentionally unused: when the ARP is active its OWN velocity wins
        // sv = this step's ABSOLUTE velocity (0.1..2.0 = 10..200 %). The ARP's per-step velocity
        // takes PRECEDENCE over how hard the note was played (#136): the step value IS the output
        // velocity (100 % = full), independent of playedVel – so an arp pattern sounds the same
        // however you touch the keys. > 100 % accents via the voice's vel->amp / vel->cutoff
        // (velocity > 1.0 is a gentle linear boost); the output safety clipper guards the bus.
        emit (pos, note, sv, true);
        activeNote = note;
        gateRemaining = std::max (1.0, (double) cfg.gate * cfg.samplesPerStep);
    }
}

void pickNote (int& outNote, float& outVel)
{
    const int n = heldCount;
    const int oct = std::max (1, std::min (4, cfg.octaves));
    const int total = std::max (1, n * oct);
    int idx = 0;
    switch (cfg.mode)
    {
        case Up:      idx = seqCursor % total; break;
        case Down:    idx = (total - 1) - (seqCursor % total); break;
        case UpDown:  { const int period = (total <= 1) ? 1 : (2 * total - 2);
                       const int p = seqCursor % period; idx = (p < total) ? p : (period - p); } break;
        case Random:  idx = (int) (nextRand() % (std::uint32_t) total); break;
        case AsPlayed: idx = seqCursor % total; break;
    }
}

```

```

        default:      idx = seqCursor % total; break;
    }
    ++seqCursor;

    const int within = idx % n;
    const int octave = idx / n;
    if (cfg.mode == AsPlayed) { outNote = heldNote[(std::size_t) within]; outVel = heldVel[(std::size_t)
within]; }
    else { outNote = sortNote[(std::size_t) within]; outVel = sortVel[(std::size_t)
within]; }
    outNote = std::min (127, std::max (0, outNote + 12 * octave));
}

void addHeld (int note, float vel)
{
    for (int i = 0; i < heldCount; ++i) if (heldNote[(std::size_t) i] == note) { heldVel[(std::size_t) i] = vel;
rebuildSorted(); return; }
    if (heldCount >= kMaxHeld) return;
    heldNote[(std::size_t) heldCount] = note;
    heldVel[(std::size_t) heldCount] = vel;
    ++heldCount;
    rebuildSorted();
}

void removeHeld (int note)
{
    for (int i = 0; i < heldCount; ++i)
        if (heldNote[(std::size_t) i] == note)
        {
            for (int j = i; j < heldCount - 1; ++j) { heldNote[(std::size_t) j] = heldNote[(std::size_t) (j +
1)]; heldVel[(std::size_t) j] = heldVel[(std::size_t) (j + 1)]; }
            --heldCount; rebuildSorted(); return;
        }
}

void rebuildSorted()
{
    for (int i = 0; i < heldCount; ++i) { sortNote[(std::size_t) i] = heldNote[(std::size_t) i];
sortVel[(std::size_t) i] = heldVel[(std::size_t) i]; }
    for (int i = 1; i < heldCount; ++i) // insertion sort by note (small n)
        for (int j = i; j > 0 && sortNote[(std::size_t) (j - 1)] > sortNote[(std::size_t) j]; --j)
            { std::swap (sortNote[(std::size_t) (j - 1)], sortNote[(std::size_t) j]); std::swap
(sortVel[(std::size_t) (j - 1)], sortVel[(std::size_t) j]); }
}

std::uint32_t nextRand() { rng ^= rng << 13; rng ^= rng >> 17; rng ^= rng << 5; return rng; }

Config cfg;
std::array<int, kMaxHeld> heldNote { };
std::array<float, kMaxHeld> heldVel { };
std::array<int, kMaxHeld> sortNote { };
std::array<float, kMaxHeld> sortVel { };
int heldCount = 0, keysDown = 0;
int activeNote = -1;
int stepIndex = -1;
double sampleInStep = 0.0, gateRemaining = 0.0;
int seqCursor = 0;
bool started = false;
std::uint32_t rng = 0x9e3779b9u;
};

```

===== FILE 5/75: Source/DSP/AudioLoop.h (136 lines) =====

// Synth — Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.

#pragma once

#include <vector>

#include <cstdint>

#include <algorithm>

#include "LoopSeam.h"

```

// =====
// Stereo AUDIO loop — JUCE-free, RT-safe on the audio thread (no alloc/lock there).
//
// The tape complement to the MIDI Looper: it records a stereo signal (the focused
// part's post-FX output, captured by the engine) into a preallocated ring, then
// plays it back summed into the master. Storage is allocated ONCE in prepare()
// (message/prepare thread); recordBlock/playBlock/advance never allocate.
//
// Record is overdub-by-sum (a fresh pass writes onto silence; later passes layer),

```

```

// matching the MIDI looper's overdub feel. The owner drives the shared loop clock:
// same loop length + one advance(n) per block keeps this lane sample-locked to the
// MIDI lane (both start at 0 and step identically).
// =====

class AudioLoop
{
public:
    // Ring-size ceiling for ONE audio lane (J2). Sized so audio loops are honest at live
    // tempos: 32 bars * 4 beats * (60 s / 120 BPM) = 64 s. So 32-bar audio records fully down
    // to ~120 BPM and 16-bar down to ~60 BPM; below that the audio lane's selectable bar count
    // is reduced (the processor's honest clamp) so the UI never promises a length the ring can't
    // hold. MIDI lanes keep full length (1..32 bars) at ANY tempo.
    // Total resident audio memory = kMaxLoopSeconds * sampleRate * 2 ch * 4 B * 4 lanes
    // (~98 MB @ 48 kHz — negligible RAM; the minimum-spec target deploy constraint is CPU %, not memory).
    static constexpr double kMaxLoopSeconds = 64.0;

    // Allocate the ring for the largest loop the transport can ask for. Call from
    // prepareToPlay (allocation is fine there, never on the audio thread).
    void prepare (int maxSamples)
    {
        cap = maxSamples < 1 ? 1 : maxSamples;
        bufL.assign ((std::size_t) cap, 0.0f);
        bufR.assign ((std::size_t) cap, 0.0f);
        loopLen = cap; pos = 0; content = false;
    }

    void setLoopLength (int samples)
    {
        loopLen = samples < 1 ? 1 : (samples > cap ? cap : samples); // never exceed the ring
        if (pos >= loopLen) pos %= loopLen;
    }

    int loopLength() const { return loopLen; }
    int position() const { return pos; }

    void setRecording (bool b) { rec = b; }
    void setPlaying (bool b) { play = b; }
    bool recording() const { return rec; }
    bool playing() const { return play; }
    bool hasContent() const { return content; }

    void clear()
    {
        std::fill (bufL.begin(), bufL.end(), 0.0f);
        std::fill (bufR.begin(), bufR.end(), 0.0f);
        pos = 0; content = false;
    }

    // Overdub this block's input into the ring at the loop position (REC only).
    void recordBlock (const float* inL, const float* inR, int n)
    {
        if (! rec || cap == 0) return;
        for (int i = 0; i < n; ++i)
        {
            const int idx = (pos + i) % loopLen;
            bufL[(std::size_t) idx] += inL[i];
            bufR[(std::size_t) idx] += inR[i];
        }
        content = true;
    }

    // Add this block of loop content into the output (PLAY only).
    void playBlock (float* outL, float* outR, int n) const
    {
        if (! play || ! content || cap == 0) return;
        // #146 Bug A (audio lane): declick the loop seam. Over the last `seam` samples the tail is
        // ramped toward the head's first sample, so the wrap (buf[len-1] -> buf[0]) is CONTINUOUS
        // with no step. Loop length is preserved exactly (tempo stays locked), and the ramp is
        // transparent on sustained/steady material (it approaches a real sample value, not silence,
        // so there is no audible dip) — inaudible on a pad, but it kills the click on any content
        // whose start and end don't match. Same seam length as the MIDI-lane note-off nudge.
        const int seam = std::min (loopseam::kSeamSamples, loopLen / 4);
        for (int i = 0; i < n; ++i)
        {
            const int idx = (pos + i) % loopLen;
            float l = bufL[(std::size_t) idx], r = bufR[(std::size_t) idx];

```

```

        if (seam > 0 && idx >= loopLen - seam)
        {
            const int k = idx - (loopLen - seam); // 0..seam-1 into the seam
            const float t = (float) (k + 1) / (float) seam; // (0,1]
            const float s = t * t * (3.0f - 2.0f * t); // smoothstep: C1, no derivative jump
            l = l * (1.0f - s) + bufL[0] * s; // ramp tail -> head[0] (buf[0])
            r = r * (1.0f - s) + bufR[0] * s;
        }
        outL[i] += l;
        outR[i] += r;
    }
}

void advance (int n) { pos = (pos + n) % loopLen; }

// Export access (message thread): the recorded content is the first loopLen samples.
int contentLength() const { return content ? loopLen : 0; }
const float* dataL() const { return bufL.data(); }
const float* dataR() const { return bufR.data(); }

// J3 scenes (message thread): snapshot the recorded region into heap buffers, and recall it.
// Compact: only the actually-recorded `contentLength()` samples are copied, so per-scene audio
// memory scales with what was recorded, not the full ring.
void snapshotInto (std::vector<float>& outL, std::vector<float>& outR) const
{
    const int len = contentLength();
    outL.assign (bufL.begin(), bufL.begin() + len);
    outR.assign (bufR.begin(), bufR.begin() + len);
}

void loadFrom (const std::vector<float>& inL, const std::vector<float>& inR)
{
    std::fill (bufL.begin(), bufL.end(), 0.0f);
    std::fill (bufR.begin(), bufR.end(), 0.0f);
    const int len = (int) std::min ({ inL.size(), inR.size(), (std::size_t) cap });
    for (int i = 0; i < len; ++i) { bufL[(std::size_t) i] = inL[(std::size_t) i]; bufR[(std::size_t) i] =
inR[(std::size_t) i]; }
    content = (len > 0);
    pos = 0;
}

private:
    std::vector<float> bufL, bufR;
    int cap = 0, loopLen = 1, pos = 0;
    bool rec = false, play = false, content = false;
};

```

===== FILE 6/75: Source/DSP/ChordEngine.h (241 lines) =====

// Synth — Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.

#pragma once

#include <array>

#include <cstdint>

```

// =====
// Diatonic chord engine. Hand-rolled, JUCE-free, allocation-free.
//
// A per-note MIDI preprocessing stage between event intake and voice allocation:
// one played note expands into a diatonic chord, so one finger = a chord that
// stays in key. Out-of-scale notes (with no forcer) pass straight through.
//
// GRAMMAR
// * In-scale note, no forcer -> the diatonic triad stacked in thirds from that
//   scale degree (root position). So in C major: C->C E G, D->D F A, ...
// * A momentary FORCER overrides the quality with absolute intervals rooted on
//   the played note (works on any note, in or out of scale):
//   MAJ 0 4 7 | MIN 0 3 7 | SUS4 0 5 7 | SUS2 0 2 7 | DIM 0 3 6 | DOM7 0 4 7 10
//   Conflicting forcercs: the latest still-held one wins (a held-order stack).
// * 7TH adds a seventh: the DIATONIC seventh when no forcer (C->Cmaj7, D->Dm7,
//   G->G7, B->Bm7b5 in C major), otherwise the seventh that fits the forcer
//   (MIN+7TH = min7, MAJ+7TH = maj7, ...). DOM7 always carries its seventh.
//
// LEDGER
// The exact tones a note produced are recorded at note-on in a fixed-size,
// note-indexed ledger. note-off replays that ledger entry, so a chord releases
// precisely the tones it triggered no matter how the modifiers changed while it
// was held. On a re-press of a held note the tones that changed are released and
// the rest retrigger (no stuck notes), so the caller feeds engine.noteOn/off.

```

```

//
// Chord OFF (disabled) is a bit-identical passthrough: one note in, one note out,
// still ledgered so note-off is symmetric.
// =====

class ChordEngine
{
public:
    enum Scale { Major = 0, Minor = 1 };
    // Forcers (mutually exclusive, latest-held wins). None = follow the scale.
    enum Forcer { None = 0, Maj, Min, Sus4, Sus2, Dim, Dom7, kNumForcers };

    static constexpr int kMaxTones = 4;      // triad + optional 7th

    // ---- configuration (message/audio thread; plain values) -----
    void setEnabled (bool e)    { enabled = e; }
    void setRoot (int pc)      { root = ((pc % 12) + 12) % 12; }
    void setScale (int s)      { scale = (s == Minor) ? Minor : Major; }
    bool isEnabled() const    { return enabled; }

    // A momentary forcer press/release. Maintains a held-order stack so the latest
    // still-pressed forcer is the active one (release it and the prior returns).
    void setForcerHeld (Forcer f, bool held)
    {
        if (f <= None || f >= kNumForcers) return;
        // remove any existing occurrence
        int w = 0;
        for (int i = 0; i < forcerCount; ++i)
            if (forcerStack[(std::size_t) i] != f) forcerStack[(std::size_t) w++] = forcerStack[(std::size_t) i];
        forcerCount = w;
        if (held && forcerCount < (int) forcerStack.size())
            forcerStack[(std::size_t) forcerCount++] = f;
    }

    void setSeventhHeld (bool held) { seventhHeld = held; }

    // Canonical modifier ids – the single bit layout shared by the QWERTY keys, the
    // learned CC/note sources, and the UI. Order is fixed (persisted / bit-packed).
    enum ModifierId { ModMaj = 0, ModMin, ModSus4, ModSus2, ModDim, ModDom7, Mod7th, kNumModifiers };

    void setModifierHeld (int modId, bool held)
    {
        switch (modId)
        {
            case ModMaj:  setForcerHeld (Maj, held); break;
            case ModMin:  setForcerHeld (Min, held); break;
            case ModSus4: setForcerHeld (Sus4, held); break;
            case ModSus2: setForcerHeld (Sus2, held); break;
            case ModDim:  setForcerHeld (Dim, held); break;
            case ModDom7: setForcerHeld (Dom7, held); break;
            case Mod7th:  setSeventhHeld (held);    break;
            default:      break;
        }
    }

    static const char* modifierName (int modId)
    {
        static const char* names[kNumModifiers] { "MAJ", "MIN", "SUS4", "SUS2", "DIM", "DOM7", "7TH" };
        return (modId >= 0 && modId < kNumModifiers) ? names[modId] : "";
    }

    Forcer activeForcer() const { return forcerCount > 0 ? forcerStack[(std::size_t) (forcerCount - 1)] : None; }
    bool seventhActive() const { return seventhHeld; }

    // ---- note events (audio thread; no allocation) -----
    // Expand a played note into its chord. Fills `trigger` with the notes to
    // engine.noteOn (all at `velocity` – caller supplies it) and `release` with any
    // prior tones on this note that are no longer part of the chord (to noteOff
    // first). Returns nothing; counts come back through the out params.
    void noteOn (int note, float velocity, int* trigger, int& nTrigger, int* release, int& nRelease)
    {
        note = clampNote (note);
        int tones[kMaxTones]; int n = buildChord (note, tones);

        // release = old tones (if this note was already sounding) not in the new set
        nRelease = 0;
        auto& e = ledger[(std::size_t) note];

```

```

    if (e.active)
        for (int i = 0; i < e.count; ++i)
            if (! contains (tones, n, e.tones[i]))
                release[nRelease++] = e.tones[i];

    // record + emit the new tones
    nTrigger = n;
    for (int i = 0; i < n; ++i) trigger[i] = tones[i];
    e.active = true; e.count = n; e.vel = velocity; // remember velocity for re-voicing (1.4)
    for (int i = 0; i < n; ++i) e.tones[i] = tones[i];
}

// 1.4: re-voice every HELD chord for the current modifier state (call after a modifier
// press/release edge). Per held trigger note, diff old vs new tones: releases removed
// tones and triggers added tones at the note's stored velocity; unchanged tones keep
// sounding (no retrigger click). Updates the ledger. RT-safe (no allocation).
// release(int note); trigger(int note, float velocity)
template <typename Release, typename Trigger>
void revoiceHeld (Release&& release, Trigger&& trigger)
{
    if (! enabled) return;
    for (int played = 0; played < 128; ++played)
    {
        auto& e = ledger[(std::size_t) played];
        if (! e.active) continue;
        int tones[kMaxTones]; const int n = buildChord (played, tones);
        for (int i = 0; i < e.count; ++i) if (! contains (tones, n, e.tones[i])) release (e.tones[i]);
        for (int i = 0; i < n; ++i) if (! contains (e.tones, e.count, tones[i])) trigger (tones[i],
e.vel);
        e.count = n; for (int i = 0; i < n; ++i) e.tones[i] = tones[i];
    }
}

// Release exactly the tones this note produced; clears the ledger entry.
void noteOff (int note, int* release, int& nRelease)
{
    note = clampNote (note);
    auto& e = ledger[(std::size_t) note];
    nRelease = 0;
    if (! e.active) { release[nRelease++] = note; return; } // safety: release the note itself
    for (int i = 0; i < e.count; ++i) release[nRelease++] = e.tones[i];
    e.active = false; e.count = 0;
}

// Forget held chords WITHOUT touching the momentary forcer stack (1.3: on an edit-focus
// hand-off the caller releases the old part's voices; drop the ledger so a later note-off
// doesn't replay tones onto the new part, but keep any still-held modifier keys).
void clearHeld() { for (auto& e : ledger) { e.active = false; e.count = 0; } }

// Forget all held chords (e.g. on disable/panic). The caller is expected to
// have released voices separately (allNotesOff).
void reset()
{
    for (auto& e : ledger) { e.active = false; e.count = 0; }
    forcerCount = 0; seventhHeld = false;
}

// Max extra tones a single note can create (for caller buffer sizing).
static constexpr int maxTonesPerNote() { return kMaxTones; }

private:
    struct Entry { bool active = false; int count = 0; int tones[kMaxTones] {}; float vel = 0.8f; };

    static int clampNote (int n) { return n < 0 ? 0 : (n > 127 ? 127 : n); }
    static bool contains (const int* a, int n, int v) { for (int i = 0; i < n; ++i) if (a[i] == v) return true;
return false; }

    const int* scaleTable() const
    {
        static constexpr int major[7] { 0, 2, 4, 5, 7, 9, 11 };
        static constexpr int minor[7] { 0, 2, 3, 5, 7, 8, 10 };
        return scale == Minor ? minor : major;
    }

    // Semitone offset of scale degree d (d may exceed 6 -> wraps octaves).
    int extScale (int d) const { return 12 * (d / 7) + scaleTable()[static_cast<int>(std::size_t) (d % 7)]; }

```

```

// Add `v` to out[] if in range and not already present. Returns updated count.
static int addTone (int* out, int count, int v)
{
    if (v < 0 || v > 127) return count;
    for (int i = 0; i < count; ++i) if (out[i] == v) return count;
    out[count] = v; return count + 1;
}

// Build the chord for a played note into out[kMaxTones]; returns the count.
int buildChord (int note, int* out) const
{
    if (! enabled) { out[0] = note; return 1; }

    const Forcer f = activeForcer();
    int count = 0;

    if (f == None)
    {
        // Diatonic: find the played note's scale degree.
        const int pc = (((note - root) % 12) + 12) % 12;
        int deg = -1;
        const int* sc = scaleTable();
        for (int k = 0; k < 7; ++k) if (sc[k] == pc) { deg = k; break; }

        if (deg < 0) { out[0] = note; return 1; } // out of scale -> single note

        count = addTone (out, count, note); // root (degree)
        count = addTone (out, count, note + extScale (deg + 2) - sc[deg]); // third
        count = addTone (out, count, note + extScale (deg + 4) - sc[deg]); // fifth
        if (seventhHeld)
            count = addTone (out, count, note + extScale (deg + 6) - sc[deg]);
    }
    else
    {
        // Forced absolute-interval chord rooted on the played note.
        static constexpr int tri[kNumForcers][3] {
            {0,0,0}, // None (unused)
            {0,4,7}, // Maj
            {0,3,7}, // Min
            {0,5,7}, // Sus4
            {0,2,7}, // Sus2
            {0,3,6}, // Dim
            {0,4,7}, // Dom7
        };
        static constexpr int seventh[kNumForcers] { 0, 11, 10, 10, 10, 9, 10 };
        for (int i = 0; i < 3; ++i) count = addTone (out, count, note + tri[f][i]);
        if (seventhHeld || f == Dom7)
            count = addTone (out, count, note + seventh[f]);
    }
    return count;
}

bool enabled = false;
int root = 0; // pitch class 0..11
int scale = Major;

std::array<Forcer, kNumForcers> forcerStack {};
int forcerCount = 0;
bool seventhHeld = false;

std::array<Entry, 128> ledger {}; // per played-note tone record
};

```

===== FILE 7/75: Source/DSP/Chorus.h (153 lines) =====

// Synth — Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.

#pragma once

#include <vector>

#include <cmath>

#include <algorithm>

```

// =====
// Stereo chorus. Hand-rolled, JUCE-free, allocation-free after prepare().
//
// A single modulated delay line per channel, the two LFOs 90° apart so a
// mono-duplicated input comes out decorrelated (wide). Delay centre ~12 ms,
// swinging ±8 ms at full depth — the classic lush chorus range. Fractional taps

```

```

// are read with linear interpolation; rate/depth/mix are smoothed so knob moves
// never zipper.
//
// VOICES (Tier 4c): with `voices == 2` a SECOND modulated tap is read per channel
// at a longer centre delay (19 ms) with its LFO inverted (180°/270°) relative to
// the first — dimension-style thickening that further decorrelates L/R. The two
// taps are summed at half weight so the wet level stays controlled. voices == 1
// (default) is the exact single-tap path (goldens bit-identical); toggling 1<->2
// crossfades via a smoothed blend (`smVoices`), so it is click-free mid-note.
//
// The delay-line buffers are sized once in prepare() (50 ms) and never resized;
// process() only reads/writes existing storage.
// =====

class Chorus
{
public:
    void prepare (double newSampleRate, int /*maxBlock*/)
    {
        sampleRate = newSampleRate;
        bufLen = (int) (newSampleRate * 0.05) + 4;      // 50 ms + guard
        bufL.assign ((size_t) bufLen, 0.0f);
        bufR.assign ((size_t) bufLen, 0.0f);
        reset();
    }

    void reset()
    {
        std::fill (bufL.begin(), bufL.end(), 0.0f);
        std::fill (bufR.begin(), bufR.end(), 0.0f);
        writePos = 0;
        phase = 0.0f;
        smMix = mix;
        smDepth = depth;
        smVoices = voices2 ? 1.0f : 0.0f;
    }

    // rateHz 0.05..8, depth 0..1, mix 0..1 (wet fraction), voices 1 or 2.
    void setParams (float rateHz, float depth01, float mix01, int voices = 1)
    {
        rate = std::clamp (rateHz, 0.05f, 8.0f);
        depth = std::clamp (depth01, 0.0f, 1.0f);
        mix = std::clamp (mix01, 0.0f, 1.0f);
        voices2 = voices >= 2;
    }

    void process (float* left, float* right, int numSamples)
    {
        const float phaseInc = rate / (float) sampleRate;
        const float centreSamp = kCentreMs * 0.001f * (float) sampleRate;
        const float swingSamp = kSwingMs * 0.001f * (float) sampleRate;

        const float centreSamp2 = kCentreMs2 * 0.001f * (float) sampleRate;
        const float voicesTarget = voices2 ? 1.0f : 0.0f;

        for (int i = 0; i < numSamples; ++i)
        {
            smMix += kSmoothCoef * (mix - smMix);
            smDepth += kSmoothCoef * (depth - smDepth);
            smVoices += kSmoothCoef * (voicesTarget - smVoices);    // dual-tap blend (click-safe toggle)

            bufL[(size_t) writePos] = left[i];
            bufR[(size_t) writePos] = right[i];

            const float lfoL = std::sin (kTwoPi * phase);
            const float lfoR = std::sin (kTwoPi * phase + kHalfPi);    // quadrature -> width

            const float dL = centreSamp + swingSamp * smDepth * lfoL;
            const float dR = centreSamp + swingSamp * smDepth * lfoR;

            float wetL = readInterp (bufL, (float) writePos - dL);
            float wetR = readInterp (bufR, (float) writePos - dR);

            if (smVoices > 0.0f)
            {
                // Second tap: longer centre delay, LFO at 120°/240° (independent of tap A's

```

```

        // 0°/90°) so L and R gain genuinely different modulation -> wider field. Blend
        // from tap-A-only (g=0) to half-A + half-B (g=1).
        const float lfoBL = std::sin(kTwoPi * phase + kThird);
        const float lfoBR = std::sin(kTwoPi * phase + kTwoThird);
        const float dL2 = centreSamp2 + swingSamp * smDepth * lfoBL;
        const float dR2 = centreSamp2 + swingSamp * smDepth * lfoBR;
        const float wetL2 = readInterp(bufL, (float) writePos - dL2);
        const float wetR2 = readInterp(bufR, (float) writePos - dR2);
        const float g = 0.5f * smVoices;
        wetL = wetL * (1.0f - g) + wetL2 * g;
        wetR = wetR * (1.0f - g) + wetR2 * g;
    }

    left[i] = left[i] * (1.0f - smMix) + wetL * smMix;
    right[i] = right[i] * (1.0f - smMix) + wetR * smMix;

    phase += phaseInc;
    if (phase >= 1.0f) phase -= 1.0f;
    if (++writePos >= bufLen) writePos = 0;
}

void copyStateFrom (const Chorus& other)
{
    // Buffers are identically sized (same prepare); copy in place – no alloc.
    std::copy(other.bufL.begin(), other.bufL.end(), bufL.begin());
    std::copy(other.bufR.begin(), other.bufR.end(), bufR.begin());
    writePos = other.writePos;
    phase = other.phase;
    smMix = other.smMix;
    smDepth = other.smDepth;
    smVoices = other.smVoices;
}

private:
    // Linear-interpolated read at a fractional (possibly negative) ring position.
    float readInterp (const std::vector<float>& buf, float pos) const
    {
        while (pos < 0.0f) pos += (float) bufLen;
        while (pos >= (float) bufLen) pos -= (float) bufLen;
        const int i0 = (int) pos;
        const int i1 = (i0 + 1 >= bufLen) ? 0 : i0 + 1;
        const float f = pos - (float) i0;
        return buf[(size_t) i0] * (1.0f - f) + buf[(size_t) i1] * f;
    }

    static constexpr float kTwoPi = 6.283185307179586f;
    static constexpr float kHalfPi = 1.5707963267948966f;
    static constexpr float kThird = 2.0943951023931953f; // 120°
    static constexpr float kTwoThird = 4.1887902047863905f; // 240°
    static constexpr float kCentreMs = 12.0f;
    static constexpr float kCentreMs2 = 19.0f; // second tap (Tier 4c) – a distinct, longer delay
    static constexpr float kSwingMs = 8.0f;
    static constexpr float kSmoothCoef = 0.001f;

    double sampleRate = 44100.0;
    std::vector<float> bufL, bufR;
    int bufLen = 0;
    int writePos = 0;
    float phase = 0.0f;

    float rate = 0.8f, depth = 0.5f, mix = 0.5f;
    bool voices2 = false;
    float smMix = 0.5f, smDepth = 0.5f, smVoices = 0.0f;
};

```

===== FILE 8/75: Source/DSP/FXChain.h (221 lines) =====

```

// Synth – Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.
#pragma once
#include "Chorus.h"
#include "StereoDelay.h"
#include "Reverb.h"
#include "StereoWidth.h"
#include "PartEQ.h"
#include <vector>
#include <cmath>

```

```

#include <algorithm>

// =====
// Per-part reorderable stereo FX chain. Hand-rolled, JUCE-free, allocation-free
// after prepare().
//
// Four REORDERABLE blocks – chorus, delay, reverb, stereo width – applied in a
// user-defined order, plus the per-part EQ as a FIXED final stage. A disabled block
// is skipped entirely (no CPU, like an oscillator kill switch), so the cost scales
// with what's actually on.
//
// K1: the per-part EQ (index 4) is no longer reorderable – it always runs LAST,
// after every other block, regardless of where EQ_ sits in order[]. Its order[] slot
// is positionally inert (kept only so old presets deserialise unchanged and the
// enable still drives the crossfade). This is the whole "one EQ at the end of the
// part's chain" model; the UI drag panel exposes only the four reorderable FX.
//
// REORDERING / TOGGLING is click-free via a ~30 ms equal-power crossfade between
// two internal chain copies: on any configuration change the standby copy inherits
// the active copy's state, adopts the new order/enables, and the two are blended
// over the window before the standby becomes active. The doubled cost only lasts
// for the fade, which happens on a user gesture – never in steady state. EQ on/off is
// an enabled[] change, so it fades in/out click-free too.
//
// Effect indices (order[] is a permutation of these):
// 0 = chorus, 1 = delay, 2 = reverb, 3 = width, 4 = EQ (4-band per-part, fixed last)
// =====

struct FXParams
{
    // Continuous params (smoothed inside each effect – no crossfade needed).
    float chorusRate = 0.8f, chorusDepth = 0.5f, chorusMix = 0.5f;
    int   chorusVoices = 1; // Tier 4c: 1 or 2 taps
    float delayTimeMs = 300.0f, delayFeedback = 0.35f, delayMix = 0.35f, delaySpread = 1.0f;
    float reverbSize = 0.5f, reverbDamp = 0.5f, reverbWidth = 1.0f, reverbMix = 0.3f, reverbMotion = 0.0f;
    float width = 1.4f;
    float sat = 0.0f; // tube saturation in the width block (0 = clean)
    float trim = 1.0f; // per-patch program level (patch_trim). NOT an effect –
                      // it rides this per-part bundle so it travels with every
                      // bake; the engine applies it post-FX in mixParts (unity

1.0).
    PartEQ::Band eqBand1 { 180.0f, 0.0f, 0.9f }; // per-part EQ, 5 fully parametric bells (fixed last)
    PartEQ::Band eqBand2 { 1000.0f, 0.0f, 0.9f };
    PartEQ::Band eqBand3 { 5000.0f, 0.0f, 0.9f };
    PartEQ::Band eqBand4 { 10000.0f, 0.0f, 0.9f };
    PartEQ::Band eqBand5 { 14000.0f, 0.0f, 0.9f };

    // Structural config (a change here triggers the crossfade).
    bool enabled[5] { false, false, false, false, false };
    int order[5] { 0, 1, 2, 3, 4 };
};

class FXChain
{
public:
    enum FX { Chorus_ = 0, Delay_ = 1, Reverb_ = 2, Width_ = 3, EQ_ = 4, kNumFX = 5 };

    void prepare (double newSampleRate, int maxBlock)
    {
        a.prepare (newSampleRate, maxBlock);
        b.prepare (newSampleRate, maxBlock);
        scratchL.assign ((size_t) maxBlock, 0.0f);
        scratchR.assign ((size_t) maxBlock, 0.0f);
        xfadeLen = std::max (1, (int) (newSampleRate * 0.030)); // 30 ms
        active = &a; standby = &b;
        reset();
    }

    void reset()
    {
        a.reset(); b.reset();
        xfadePos = 0;
        crossfading = false;
    }

    // Push the full parameter set. Continuous params go to both copies every call

```

```

// (they smooth internally). A change to order/enables is NOT applied to the
// active chain here – process() detects it and crossfades it in, so the active
// config only ever changes at a fade boundary. Applying it eagerly would
// hard-switch the routing and click.
void setParams (const FXParams& p)
{
    desired = p;
    a.setContinuous (p);
    b.setContinuous (p);
}

void process (float* left, float* right, int numSamples)
{
    if (! crossfading && configDiffers (active->order, active->enabled, desired.order, desired.enabled))
        startCrossfade();

    if (! crossfading)
    {
        active->process (left, right, numSamples);
        return;
    }

    // Crossfade: A in place, B on a copy of the same input, then blend.
    std::copy (left, left + numSamples, scratchL.begin());
    std::copy (right, right + numSamples, scratchR.begin());
    active->process (left, right, numSamples);
    standby->process (scratchL.data(), scratchR.data(), numSamples);

    for (int i = 0; i < numSamples; ++i)
    {
        const float t = std::min (1.0f, (float) (xfadePos + i) / (float) xfadeLen);
        const float gA = std::cos (t * kHalfPi); // equal-power
        const float gB = std::sin (t * kHalfPi);
        left[i] = left[i] * gA + scratchL[(size_t) i] * gB;
        right[i] = right[i] * gA + scratchR[(size_t) i] * gB;
    }

    xfadePos += numSamples;
    if (xfadePos >= xfadeLen)
    {
        std::swap (active, standby); // B is now the live chain
        crossfading = false;
    }
}

bool isCrossfading() const { return crossfading; }

private:
// One full copy of the five effects plus its own order/enable config.
struct Chain
{
    Chorus chorus; StereoDelay delay; Reverb reverb; StereoWidth width; PartEQ eq;
    int order[kNumFX] { 0, 1, 2, 3, 4 };
    bool enabled[kNumFX] { false, false, false, false, false };

    void prepare (double sr, int maxBlock)
    {
        chorus.prepare (sr, maxBlock);
        delay.prepare (sr, maxBlock);
        reverb.prepare (sr);
        width.prepare (sr);
        eq.prepare (sr);
    }

    void reset() { chorus.reset(); delay.reset(); reverb.reset(); width.reset(); eq.reset(); }

    void setContinuous (const FXParams& p)
    {
        chorus.setParams (p.chorusRate, p.chorusDepth, p.chorusMix, p.chorusVoices);
        delay.setParams (p.delayTimeMs, p.delayFeedback, p.delayMix, p.delaySpread);
        reverb.setParams (p.reverbSize, p.reverbDamp, p.reverbWidth, p.reverbMix, p.reverbMotion);
        width.setWidth (p.width);
        width.setSat (p.sat);
        eq.setBands (p.eqBand1, p.eqBand2, p.eqBand3, p.eqBand4, p.eqBand5);
    }

    void setConfig (const int ord[kNumFX], const bool en[kNumFX])
    {

```

```

        for (int i = 0; i < kNumFX; ++i) { order[i] = ord[i]; enabled[i] = en[i]; }
    }
    void copyStateFrom (const Chain& o)
    {
        chorus.copyStateFrom (o.chorus);
        delay.copyStateFrom (o.delay);
        reverb.copyStateFrom (o.reverb);
        width.copyStateFrom (o.width);
        eq.copyStateFrom (o.eq);
    }

    void process (float* L, float* R, int n)
    {
        for (int slot = 0; slot < kNumFX; ++slot)
        {
            const int fx = order[slot];
            if (fx < 0 || fx >= kNumFX || ! enabled[fx]) continue;    // skipped -> free
            switch (fx)
            {
                case Chorus_: chorus.process (L, R, n); break;
                case Delay_: delay.process (L, R, n); break;
                case Reverb_: reverb.process (L, R, n); break;
                case Width_: width.process (L, R, n); break;
                case EQ_: break;    // K1: EQ is not reorderable – applied last, below
                default: break;
            }
        }
        // Fixed final stage: the per-part EQ always runs after everything else,
        // ignoring its position in order[] (its enable still gates it click-free).
        if (enabled[EQ_]) eq.process (L, R, n);
    }
};

void startCrossfade()
{
    standby->copyStateFrom (*active);
    standby->setContinuous (desired);
    standby->setConfig (desired.order, desired.enabled);
    xfadePos = 0;
    crossfading = true;
}

static bool configDiffers (const int oA[kNumFX], const bool eA[kNumFX],
                          const int oB[kNumFX], const bool eB[kNumFX])
{
    for (int i = 0; i < kNumFX; ++i)
        if (oA[i] != oB[i] || eA[i] != eB[i]) return true;
    return false;
}

static constexpr float kHalfPi = 1.5707963267948966f;

Chain a, b;
Chain* active = &a;
Chain* standby = &b;
std::vector<float> scratchL, scratchR;
int xfadeLen = 1440;
int xfadePos = 0;
bool crossfading = false;
FXParams desired;
};

```

===== FILE 9/75: Source/DSP/Kit.h (49 lines) =====

// Synth – Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.

#pragma once

#include "SynthVoice.h" // VoiceParams

#include <array>

```

// =====
// Kit part data (JUICE-free POD). A Kit part turns the paramsFor(part, note) seam
// per-note: each of up to 16 pads has its own baked VoiceParams (from a source
// preset, with its level folded into VoiceParams::gain), a trigger note, up to 4
// sounding notes (decoupling pitch from the trigger – 2..4 = a chord pad), and a
// choke group. All fixed-size; copied verbatim onto the audio thread.
// =====

```

```

static constexpr int kMaxKitPads      = 16;
static constexpr int kMaxPadSoundNotes = 4;

struct KitPad
{
    int  triggerNote = -1;                // -1 = empty pad (silent)
    int  soundNote[kMaxPadSoundNotes] = { 60, 0, 0, 0 }; // sounding pitches this pad plays
    int  numSound    = 1;                // 1 = single hit, 2..4 = chord pad
    int  chokeGroup  = 0;                // 0 = none; same nonzero group = mutually choking

    // I2: a pad can play a SAMPLE instead of the synth VoiceParams. The pointers are BORROWED
    // from the processor's SampleStore (immutable, never-freed) – this POD is copied verbatim
    // onto the audio thread, so it must carry only the borrowed pointers + metadata, never audio.
    bool    isSample    = false;
    const float* sampleL = nullptr;
    const float* sampleR = nullptr;
    int      sampleLen  = 0;
    double    sampleSR   = 48000.0;
    int       sampleRoot = 60;                // note at which ratio == 1.0
    float     sampleGain = 1.0f;              // pad level folded in (velocity applied at trigger)
};

```

```

struct KitData
{
    bool    isKit = false;
    std::array<KitPad, kMaxKitPads> pads {};
    std::array<VoiceParams, kMaxKitPads> params {}; // pad[i]'s baked voice params (gain = pad level)

    // Pad index whose trigger note == `note`, or -1 if none maps.
    int padForTrigger (int note) const
    {
        for (int i = 0; i < kMaxKitPads; ++i)
            if (pads[(std::size_t) i].triggerNote == note) return i;
        return -1;
    }
};

```

===== FILE 10/75: Source/DSP/LFO.h (77 lines) =====

// Synth – Copyright © 2026 John L Farmer. Licensed under AGPLV3; see LICENSE.

```

#pragma once
#include <cmath>
#include <random>

```

```

// =====
// LFO. Global (one instance in the engine, shared by all voices) – this
// mirrors classic polysynths where a single LFO modulates every voice in
// phase, giving that cohesive "the whole synth is wobbling" character.
// Per-voice LFOs (for e.g. free-running vibrato per note) are a v2 option.
//
// Runs at control rate: updated once per block or every N samples, not
// per-sample. LFO destinations change slowly enough that this is inaudible
// and it keeps CPU essentially free.
//
// TODO v2: MIDI clock sync (rate as note divisions) – hooks into the
// DAW / footswitch rig for tempo-locked filter wobble.
// =====

```

```

class LFO
{
public:
    enum class Shape { Triangle, Sine, Square, SampleHold };

    void prepare (double newSampleRate)
    {
        sampleRate = newSampleRate;
        phase = 0.0;
    }

    void setRate (double hz) { phaseInc = hz / sampleRate; }
    void setShape (Shape s)  { shape = s; }
    double currentPhase() const { return phase; } // for freezing on sync-off

    // Advance by `numSamples` and return the current bipolar value [-1, 1] (free-running).
    float advance (int numSamples)
    {
        const double oldPhase = phase;

```

```

    phase += phaseInc * numSamples;
    while (phase >= 1.0)
        phase -= 1.0;
    return valueForWrap (oldPhase);
}

// J1: set the phase ABSOLUTELY (for a tempo-synced, transport-position-derived LF0) and return
// its value. S&H latches a new value when the phase wraps past a cycle boundary.
float setPhase (double phase01)
{
    const double oldPhase = phase;
    phase = phase01 - std::floor (phase01);
    return valueForWrap (oldPhase);
}

private:
float valueForWrap (double oldPhase)
{
    switch (shape)
    {
        case Shape::Sine:    return static_cast<float> (std::sin (phase * 6.283185307179586));
        case Shape::Triangle: return static_cast<float> (4.0 * std::abs (phase - 0.5) - 1.0);
        case Shape::Square:  return phase < 0.5 ? 1.0f : -1.0f;
        case Shape::SampleHold:
            if (phase < oldPhase)                // wrapped -> new random value
                heldValue = dist (rng);
            return heldValue;
    }
    return 0.0f;
}

Shape shape = Shape::Triangle;
double sampleRate = 44100.0;
double phase = 0.0, phaseInc = 0.0;

std::minstd_rand rng { 0x5EED };
std::uniform_real_distribution<float> dist { -1.0f, 1.0f };
float heldValue = 0.0f;
};

```

===== FILE 11/75: Source/DSP/LoopSeam.h (22 lines) =====

// Synth — Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.
#pragma once

```

// =====
// #146 Bug A — MIDI loop-seam declick. A note held to the loop end has its synthesized
// note-off (Looper::closeDangling) nudged this many samples EARLY, so its release envelope
// doesn't collide with the next pass's re-attack at the wrap (the seam click the user heard
// as "a note playing when the loop ends has a hard stop"). The note's own release does the
// fade — no new fade path. Inaudible on a slow-release pad; softens a fast pluck's transient.
// ~5 ms @ 48 kHz.
//
// AUDIO lane (AudioLoop::playBlock): over the last kSeamSamples the loop tail is ramped toward the
// head's first sample, so the circular-buffer wrap is continuous with no step. Loop length is
// preserved exactly (tempo locked); the ramp approaches a real sample value, not silence, so
// steady/sustained material passes through transparently (no dip) while any start/end mismatch is
// declicked. One shared constant, one ramp to trust.
// =====

```

```

namespace loopseam
{
    constexpr int kSeamSamples = 240;
}

```

===== FILE 12/75: Source/DSP/Looper.h (224 lines) =====

// Synth — Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.
#pragma once

```

#include <array>
#include "../Observability/MidiTracer.h"
#include "LoopSeam.h"

```

```

// =====
// Per-part MIDI looper — JUCE-free, RT-safe (fixed storage, no alloc/lock).
//
// A tape-style loop of MIDI note events per part. The owner (the processor) plays
// back this block's events at block start, records incoming performance notes into

```

```

// the loop while REC is armed, then advances the loop clock by the block length.
//
// First-pass double-trigger is avoided by ARMING: a freshly recorded event is not
// played until the loop next wraps (the live note already sounded this pass); on
// each wrap every event in that lane is armed, so overdubs join on the following cycle.
// Playback timing is block-granular (consistent with the synth's routed-note path);
// sample-accurate loop playback is a future refinement.
//
// J2 – PER-LANE LENGTH. Each lane (== part) has its OWN loop length. A single master
// position advances the clock; a lane's effective position is masterPos % laneLen.
// Because every length is a power-of-two multiple of a bar (1..32) and the master
// wraps at the longest possible length (32 bars, a common multiple of all of them),
// every lane stays phase-aligned to the same downbeat with no drift and no reset
// pulses – a 2-bar lane simply wraps 4x inside an 8-bar lane, always on the beat.
// =====

class Looper
{
public:
    static constexpr int kParts      = 4;
    static constexpr int kMaxEvents = 512; // per part

    struct Event { int t; int note; float vel; bool on; bool armed; };
    struct Lane { std::array<Event, kMaxEvents> ev { }; int count = 0; }; // one lane's recorded clip

    // J3 scenes: snapshot / restore a lane's recorded clip (a plain copy of its events).
    const Lane& laneContent (int part) const
    { return parts[(std::size_t) (part < 0 ? 0 : part >= kParts ? kParts - 1 : part)]; }
    void setLaneContent (int part, const Lane& l) { if (part >= 0 && part < kParts) parts[(std::size_t) part] = l; }

    // The master wrap period – a common multiple of every lane length so masterPos % laneLen
    // is continuous across the wrap. Set from tempo each block (32 bars); clamped >= 1.
    void setMasterLength (int samples)
    {
        masterLen = samples < 1 ? 1 : samples;
        if (loopPos >= masterLen) loopPos %= masterLen;
    }

    // Per-lane length (samples). Each lane divides masterLen (32 / {1,2,4,8,16,32} bars).
    void setLoopLength (int part, int samples)
    {
        if (part < 0 || part >= kParts) return;
        loopLen[(std::size_t) part] = samples < 1 ? 1 : samples;
    }
    int loopLength (int part) const { return (part >= 0 && part < kParts) ? loopLen[(std::size_t) part] : 1; }

    int position() const { return loopPos; } // master (arp/seq bar anchor)
    int position (int part) const { return loopPos; } // this lane's phase
    { return (part >= 0 && part < kParts) ? loopPos % loopLen[(std::size_t) part] : 0; }

    // Per-lane transport: lane N (== part N) has its OWN REC/PLAY, so each part records and
    // plays independently and NOTHING here depends on the edit/play focus (task #47).
    void setRecording (int part, bool b)
    {
        if (part < 0 || part >= kParts) return;
        const bool was = recording_[(std::size_t) part];
        recording_[(std::size_t) part] = b;
        if (was && ! b) closeDangling (part); // stop: any note held across the stop gets a note-off
    }
    void setPlaying (int part, bool b) { if (part >= 0 && part < kParts) playing_[(std::size_t) part] = b; }
    bool recording (int part) const { return part >= 0 && part < kParts && recording_[(std::size_t) part]; }
    bool playing (int part) const { return part >= 0 && part < kParts && playing_[(std::size_t) part]; }
    bool anyPlaying() const { for (bool b : playing_) if (b) return true; return false; }

    // J3: restart every lane from its downbeat (scene switch) – position 0, all events armed so a
    // freshly-activated scene plays from the top rather than resuming mid-loop.
    void rewind() { loopPos = 0; for (int p = 0; p < kParts; ++p) armLane (p); }

    void clear (int part) { if (part >= 0 && part < kParts) { parts[(std::size_t) part].count = 0;
recHeld_[(std::size_t) part].fill (false); } } // wipe ONE lane
    void clearAll() { for (auto& p : parts) p.count = 0; for (auto& r : recHeld_) r.fill (false); loopPos = 0; }
    void reset() { clearAll(); for (int i = 0; i < kParts; ++i) { recording_[(std::size_t) i] = false;
playing_[(std::size_t) i] = false; } }

    bool hasContent (int part) const { return part >= 0 && part < kParts && parts[(std::size_t) part].count > 0; }
    int eventCount (int part) const { return (part >= 0 && part < kParts) ? parts[(std::size_t) part].count : 0; }

```

```

const Event& event (int part, int i) const { return parts[(std::size_t) part].ev[(std::size_t) i]; }

// Quantize (task #53): snap recorded note timestamps to a grid (1/32 = samplesPerStep/2).
// Per-lane on/off; the grid is shared (from tempo). Default on.
void setQuantizeGrid (int gridSamples) { quantGrid = gridSamples > 0 ? gridSamples : 0; }
void setQuantize (int part, bool on) { if (part >= 0 && part < kParts) quantize_[(std::size_t) part] = on; }
bool quantize (int part) const { return part >= 0 && part < kParts && quantize_[(std::size_t) part]; }

// Record an incoming performance note at (lane position + block offset). Lane N records
// only while ITS OWN REC is armed (part is the note's routed part – fixed, not focus).
void recordNote (int part, int offsetInBlock, int note, float vel, bool on)
{
    if (part < 0 || part >= kParts) return;
    // Track the live-held key set for THIS part, always – even while not recording. A note struck
    // before REC engages must still be known at the downbeat (captureHeldAtStart injects it at t=0),
    // else the first note is skipped. Cleared by its own note-off. Velocity kept for a faithful re-hit.
    if (note >= 0 && note < 128) heldVel_[(std::size_t) part][(std::size_t) note] = on ? (vel > 0.0f ? vel :
1.0f) : 0.0f;
    if (! recording_[(std::size_t) part]) return;
    auto& p = parts[(std::size_t) part];
    if (p.count >= kMaxEvents) return;
    const int len = loopLen[(std::size_t) part];
    int t = (position (part) + (offsetInBlock < 0 ? 0 : offsetInBlock)) % len;
    if (quantize_[(std::size_t) part] && quantGrid > 0 && note >= 0 && note < 128)
    {
        const int g = quantGrid;
        t = (((t + g / 2) / g) * g) % len; // snap to the nearest grid line
        if (on) lastOnT[(std::size_t) part][(std::size_t) note] = t;
        else if (lastOnT[(std::size_t) part][(std::size_t) note] == t)
            t = (t + g) % len; // note-off snapped onto its on -> +1 grid
    }
    (no zero-length)
    p.ev[(std::size_t) p.count++] = { t, note, vel, on, false };
    if (note >= 0 && note < 128) recHeld_[(std::size_t) part][(std::size_t) note] = on; // track
held-across-stop
    mtrace::emit (mtrace::Ev::LoopRec, part, note, (int) (vel * 127.0f), on ? 1 : 0);
}

// A lane whose record just completed plays back from THE SAME downbeat this block (no dead cycle):
// its next playBlock widens the window to start at t=0 so the just-captured downbeat is emitted,
// not dropped and heard a full cycle later. One-shot; cleared once consumed.
void requestDownbeatCatchUp (int part) { if (part >= 0 && part < kParts) catchUp_[(std::size_t) part] = true; }

// Called once when a lane's record ENGAGES at its downbeat: any key already held (struck before
// the downbeat) is captured as a note-on at t=0 – the note-on mirror of closeDangling. Without
// this, a note held across the loop start lands nowhere (its note-on arrived while this lane
// wasn't recording) and the first note is skipped unless struck exactly on the grid. recHeld_ is
// set so record-stop's closeDangling pairs a note-off; skip notes already recorded-on this pass.
void captureHeldAtStart (int part)
{
    if (part < 0 || part >= kParts || ! recording_[(std::size_t) part]) return;
    auto& p = parts[(std::size_t) part];
    for (int n = 0; n < 128; ++n)
        if (heldVel_[(std::size_t) part][(std::size_t) n] > 0.0f && ! recHeld_[(std::size_t) part][(std::size_t)
n])
        {
            if (p.count >= kMaxEvents) break;
            const float v = heldVel_[(std::size_t) part][(std::size_t) n];
            p.ev[(std::size_t) p.count++] = { 0, n, v, true, false };
            recHeld_[(std::size_t) part][(std::size_t) n] = true;
            mtrace::emit (mtrace::Ev::LoopRec, part, n, (int) (v * 127.0f), 1);
        }
}

// Emit this block's armed playback events for every PLAYING lane, each against its OWN length.
// emit(part, note, vel, on).
template <typename Emit>
void playBlock (int numSamples, Emit&& emit)
{
    for (int part = 0; part < kParts; ++part)
    {
        if (! playing_[(std::size_t) part]) continue; // this lane is stopped
        const int len = loopLen[(std::size_t) part];
        // Normally the window is [pos, pos+n). On the record->play handoff, catch the downbeat by
        // starting the window at 0 (emit everything from the loop top up to the window end, once).
        const bool catchUp = catchUp_[(std::size_t) part];

```

```

        catchUp_[(std::size_t) part] = false;
        const int start = catchUp ? 0 : (loopPos % len), end = (loopPos % len) + numSamples;
        const auto& p = parts[(std::size_t) part];
        for (int i = 0; i < p.count; ++i)
        {
            const auto& e = p.ev[(std::size_t) i];
            if (! e.armed) continue;
            const bool inWin = (end <= len) ? (e.t >= start && e.t < end)
                : (e.t >= start || e.t < end - len);
            if (inWin) { mtrace::emit (mtrace::Ev::LoopEmit, part, e.note, e.on ? 1 : 0, e.t); emit (part,
e.note, e.vel, e.on); }
        }
    }

    // Advance the master clock; on a lane's OWN wrap, arm that lane's events for the next cycle.
    void advance (int numSamples)
    {
        for (int part = 0; part < kParts; ++part)
        {
            const int len = loopLen[(std::size_t) part];
            if ((loopPos % len) + numSamples >= len) armLane (part);    // this lane wrapped
        }
        loopPos += numSamples;
        if (loopPos >= masterLen) loopPos %= masterLen;
    }

private:
    void armLane (int part)
    {
        auto& p = parts[(std::size_t) part];
        for (int i = 0; i < p.count; ++i) p.ev[(std::size_t) i].armed = true;
        if (p.count > 0) mtrace::emit (mtrace::Ev::LoopWrap, part, p.count);
    }

    // #138 root-cause fix: a note still held when recording STOPS never got its note-off recorded,
    // so the lane would replay an unbalanced note-on every pass (a stuck/accumulating trigger).
    // Synthesize the missing note-off at the stop position so every recorded note-on is paired.
    void closeDangling (int part)
    {
        auto& p = parts[(std::size_t) part];
        // #146 Bug A: place the synthesized note-off a few ms BEFORE the stop position so a note held
        // to the loop end releases before the seam – its release envelope no longer collides with the
        // next pass's re-attack at the wrap (the seam click). Inaudible on a slow pad; softens a pluck.
        const int t = std::max (0, position (part) - loopseam::kSeamSamples);
        for (int n = 0; n < 128; ++n)
            if (recHeld_[(std::size_t) part][(std::size_t) n])
            {
                recHeld_[(std::size_t) part][(std::size_t) n] = false;
                if (p.count >= kMaxEvents) continue;    // full (rare) – nothing we can append
                p.ev[(std::size_t) p.count++] = { t, n, 0.0f, false, false };
                mtrace::emit (mtrace::Ev::LoopRec, part, n, 0, 0);    // trace the synthesized close
            }
    }

    std::array<Lane, kParts> parts { };
    std::array<int, kParts> loopLen { { 48000, 48000, 48000, 48000 } };    // per-lane length (samples)
    int masterLen = 48000;    // master wrap period (32 bars)
    int loopPos = 0;    // master position [0, masterLen)

    std::array<bool, kParts> recording_ { };
    std::array<bool, kParts> playing_ { };
    std::array<bool, kParts> catchUp_ { };    // one-shot: play from the downbeat on record->play handoff
    std::array<bool, kParts> quantize_ { { true, true, true, true } };    // per-lane 1/32 quantize (default on)
    int quantGrid = 0;    // grid in samples (0 = off; set from
tempo)
    std::array<std::array<int, 128>, kParts> lastOnT { };    // snapped pos of last note-on (pairing
guard)
    std::array<std::array<bool, 128>, kParts> recHeld_ { };    // notes held mid-record (closed on record
stop, #138)
    std::array<std::array<float, 128>, kParts> heldVel_ { };    // live-held key velocities per part (0 =
up); for captureHeldAtStart
};

===== FILE 13/75: Source/DSP/MidiClock.h (71 lines) =====
// Synth – Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.
#pragma once

```

```

#include <cmath>
#include <cstdlib>

// =====
// MIDI clock generator (JUCE-free, RT-safe). Turns the transport beat position into a
// stream of 24-ppq clock ticks plus Start/Stop, at SAMPLE-ACCURATE offsets within each block so
// downstream loopers/pedals see minimal jitter.
//
// The owner calls process() once per block with the block-start beat position and samples-per-beat
// (from the J1 transport), and an emit(sampleOffset, kind) callback. Clock ticks are emitted
// whenever `enabled` (so a slave always tracks tempo); Start/Stop fire on the `running` edges.
// A transport jump (host loop/seek) re-aligns the tick phase instead of spraying a burst.
// =====

class MidiClockGenerator
{
public:
    enum Kind { Clock = 0, Start = 1, Stop = 2 };
    static constexpr double kPpq = 24.0; // MIDI clock: 24 pulses per quarter note

    void reset() { nextTick = 0; runningPrev = false; primed = false; }

    // emit(int sampleOffset, Kind). `beatsAtBlockStart` is the transport position (quarter notes)
    // at the first sample of the block; `samplesPerBeat` = sr*60/bpm; `numSamples` the block length.
    template <typename Emit>
    void process (double beatsAtBlockStart, double samplesPerBeat, int numSamples,
                 bool enabled, bool running, Emit&& emit)
    {
        if (! enabled)
        {
            if (runningPrev) { emit (0, Stop); runningPrev = false; }
            primed = false;
            return;
        }
        if (samplesPerBeat < 1.0 || numSamples <= 0) return;

        // Phase-lock the tick counter to the transport: prime on (re)enable, and re-align on a jump
        // (a host loop brace / locate moves beatsAtBlockStart discontinuously).
        const long curTick = (long) std::floor (beatsAtBlockStart * kPpq);
        if (! primed || std::labs (nextTick - curTick) > 2) { nextTick = curTick; primed = true; }

        if (running && ! runningPrev) emit (0, Start);
        if (! running && runningPrev) emit (0, Stop);
        runningPrev = running;
        if (! running) return; // clock ticks only while the transport is running

        // Reference each tick to its ABSOLUTE sample (round once) and subtract the block-start
        // sample — avoids the FP error of a small (tickBeat - blockStart) difference, so the grid is
        // exact at integer tempi and within a sample otherwise.
        const long blockStartSample = std::lround (beatsAtBlockStart * samplesPerBeat);
        const double blockEndBeats = beatsAtBlockStart + (double) numSamples / samplesPerBeat;
        while ((double) nextTick / kPpq < blockEndBeats)
        {
            const double tickBeat = (double) nextTick / kPpq;
            if (tickBeat >= beatsAtBlockStart)
            {
                int off = (int) (std::lround (tickBeat * samplesPerBeat) - blockStartSample);
                off = off < 0 ? 0 : (off >= numSamples ? numSamples - 1 : off);
                emit (off, Clock);
            }
            ++nextTick;
        }
    }
};

private:
    long nextTick = 0; // absolute 24-ppq tick index of the next clock pulse
    bool runningPrev = false;
    bool primed = false;
};

===== FILE 14/75: Source/DSP/ModMatrix.h (191 lines) =====
// Synth — Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.
#pragma once
#include <array>
#include <algorithm>
#include <cstdint>

```

```
// =====
// Mod matrix – JUCE-free, RT-safe (fixed storage, no alloc/lock/branch-on-string).
//
// A small routing table: kSlots independent slots, each { source -> dest, depth }.
// The engine fills a ModSources snapshot with the CURRENT value of every source
// (per part for LFO/wheel/macros, per voice for env/velocity/note/random), then a
// voice calls evaluate() once per control chunk to get the summed offset for each
// destination. Depth is bipolar (-1..1); at |depth|=1 and a full-scale source the
// destination moves by its documented range (kRange* below).
//
// Additive layer: this sits ON TOP of the fixed LFO dests, the env->pitch/cutoff
// routes and the velocity routes – it never replaces them. An all-inert matrix
// (every slot source/dest None, or depth 0) yields all-zero offsets, so a voice
// renders bit-identically (the golden guarantee). active() lets the caller skip
// the whole evaluation on the fast path.
// =====

// Current value of every modulation source, sampled at a control-chunk boundary.
// Per-part fields (lfo/modWheel/pitchBend/macro) are filled by the engine; per-voice
// fields (modEnv/ampEnv/velocity/noteNorm/random) by the voice.
struct ModSources
{
    float lfo[3]    { 0.0f, 0.0f, 0.0f }; // -1..1 raw LFO output
    float modEnv    = 0.0f;               // 0..1 filter/mod envelope level
    float ampEnv    = 0.0f;               // 0..1 amp envelope level
    float velocity  = 0.0f;               // 0..1 note velocity
    float noteNorm  = 0.0f;               // (note-60)/60, ~ -1..1 keytrack
    float modWheel  = 0.0f;               // 0..1
    float pitchBend = 0.0f;               // -1..1
    float random    = 0.0f;               // -1..1 per-note sample & hold
    float macro[8] { };                  // 0..1 each
};

class ModMatrix
{
public:
    static constexpr int kSlots = 8;

    // Persisted as ints – order is STABLE, only append.
    enum Source { SrcNone = 0, LFO1, LFO2, LFO3, ModEnv, AmpEnv, Velocity, Note,
        ModWheel, PitchBend, Random, Macro1, Macro2, Macro3, Macro4,
        Macro5, Macro6, Macro7, Macro8, kNumSources };
    // WavePos is RESERVED for the (not-yet-built) wavetable oscillator: it persists and
    // evaluates, but no voice seam consumes it yet, so a route to it is silent for now.
    //
    // VOICE-TIER dests (Pitch..Osc3Level) are applied per-voice inside SynthVoice via evaluate().
    // BLOCK-TIER dests (>= kFirstBlockDest) are applied by the PROCESSOR at block rate to the
    // focused part's FXParams/PartLfos/VoiceParams before they reach the engine – they modulate
    // params that aren't per-voice offsets (FX, EQ, LFO rate/depth, env stages, osc tune, ...).
    // Order is STABLE – only append. Registry metadata (paramId/name/category) lives in the
    // processor; the matrix only sums source*depth per dest id.
    enum Dest { DstNone = 0, Pitch, Cutoff, Resonance, Pulsewidth, Amp,
        WavePos, Osc1Level, Osc2Level, Osc3Level,
        // --- block-tier (append only) ---
        ChorusRate, ChorusDepth, ChorusMix,
        DelayTime, DelayFeedback, DelayMix, DelaySpread,
        ReverbSize, ReverbDamp, ReverbWidth, ReverbMix,
        StereoWidth,
        EqB1Gain, EqB2Gain, EqB3Gain,
        Lfo1Rate, Lfo1Depth, Lfo2Rate, Lfo2Depth, Lfo3Rate, Lfo3Depth,
        AmpAttack, AmpDecay, AmpSustain, AmpRelease,
        FltAttack, FltDecay, FltSustain, FltRelease,
        FilterEnvAmt, FilterKeytrack, VelToCutoff, VelToAmp, FltEnvToPitch,
        Osc1Octave, Osc1Detune, Osc2Octave, Osc2Detune, Osc3Octave, Osc3Detune,
        GlideTime, PartLevel, PartPan,
        EqB4Gain, // K1: appended (stable-order rule) though it groups with EqB1-3Gain
        EqB5Gain, // K1 tweak: 5th EQ band gain (also appended)
        NoiseLevel, // 4th-source noise level. APPENDED for state stability, but VOICE-applied
        // (evaluate() fills Offsets.noiseLevel; the voice reads it) – not a block
        // dest despite its numeric position, so blockOffsets() collects it harmlessly.
        ReverbMotion, // Tier 4a: reverb tail modulation depth (block-tier, appended).
        Saturation, // FX SAT: tube saturation in the width block (block-tier, appended).
        PartTrim, // Per-patch program level (post-FX / pre-part-level gain). Registry-listed
        // for LINK/animation; applied by the processor (setPartTrims) like PartLevel,
        // not live-summed here (shares the mixer-tier seam reserved with PartLevel/Pan).
```

```

    Osc1Fm, Osc2Fm, // #132 FM chain depth (osc2->osc1, osc3->osc2). APPENDED for state
                    // stability but VOICE-applied (evaluate() fills Offsets.osc1Fm/osc2Fm; the
                    // voice reads them) – like NoiseLevel, NOT block dests despite position, so
                    // blockOffsets() collects them harmlessly (no mod() entry consumes them).
    kNumDests };
static constexpr int kFirstBlockDest = ChorusRate;
static constexpr int kNumBlockDests = kNumDests - kFirstBlockDest;

struct Slot { int source = SrcNone; int dest = DstNone; float depth = 0.0f; }; // depth -1..1

// Summed per-destination offsets from all live slots.
struct Offsets
{
    float pitchSemis = 0.0f; // added to the pitch exponent (semitones)
    float cutoffOct = 0.0f; // added to the cutoff exponent (octaves)
    float reso = 0.0f; // added to resonance (0..1 domain)
    float pw = 0.0f; // added to pulse width
    float amp = 0.0f; // amp multiplier is clamp(1 + amp, 0, 2)
    float wavePos = 0.0f; // RESERVED (wavetable position) – no seam consumes it yet
    float osc1Level = 0.0f; // added to osc 1/2/3 level (0..1 domain)
    float osc2Level = 0.0f;
    float osc3Level = 0.0f;
    float noiseLevel = 0.0f; // added to noise level (0..1 domain)
    float osc1Fm = 0.0f; // #132 added to osc1/osc2 FM depth (0..1 domain)
    float osc2Fm = 0.0f;
};

// Full-scale destination ranges at |depth| = 1 and a full-scale source.
static constexpr float kRangePitch = 24.0f; // +/- 2 octaves
static constexpr float kRangeCutoff = 4.0f; // +/- 4 octaves
static constexpr float kRangeReso = 1.0f; // +/- full resonance
static constexpr float kRangePw = 0.45f; // matches the LFO->PW range
static constexpr float kRangeAmp = 1.0f; // +/- full (mul 0..2)
static constexpr float kRangeWavePos = 1.0f; // +/- full wavetable sweep (reserved)
static constexpr float kRangeOscLevel = 1.0f; // +/- full source level
static constexpr float kRangeFm = 1.0f; // #132 +/- full FM depth (velocity -> FM is the headline)

std::array<Slot, kSlots> slots { };

// Any live routing? Lets the voice/engine skip evaluation entirely (bit-identical).
bool active() const
{
    for (auto& s : slots)
        if (s.source != SrcNone && s.dest != DstNone && s.depth != 0.0f) return true;
    return false;
}

// Block-tier: sum the NORMALIZED (0..1 param-space) offset for each block dest into
// out[dest - kFirstBlockDest]. The processor adds these to the focused part's param values.
// out must have kNumBlockDests entries. Voice-tier dests are ignored here.
void blockOffsets (const ModSources& s, float* out, int n) const
{
    for (int i = 0; i < n; ++i) out[i] = 0.0f;
    for (auto& sl : slots)
    {
        if (sl.source == SrcNone || sl.dest < kFirstBlockDest || sl.depth == 0.0f) continue;
        const int idx = sl.dest - kFirstBlockDest;
        if (idx >= 0 && idx < n) out[idx] += sourceValue (sl.source, s) * sl.depth; // -1..1 normalized
    }
}

Offsets evaluate (const ModSources& s) const
{
    Offsets o;
    for (auto& sl : slots)
    {
        if (sl.source == SrcNone || sl.dest == DstNone || sl.depth == 0.0f) continue;
        const float v = sourceValue (sl.source, s) * sl.depth;
        switch (sl.dest)
        {
            case Pitch: o.pitchSemis += v * kRangePitch; break;
            case Cutoff: o.cutoffOct += v * kRangeCutoff; break;
            case Resonance: o.reso += v * kRangeReso; break;
            case PulseWidth: o.pw += v * kRangePw; break;
            case Amp: o.amp += v * kRangeAmp; break;
            case WavePos: o.wavePos += v * kRangeWavePos; break; // reserved
        }
    }
}

```

— *pages 26–263 omitted (middle of listing)* —

```

// The 8 macros live in the top bar (compact, with the preset + master). The
// native title-bar gesture issue is handled by running fullscreen/kiosk (no title
// bar) — see toggleFullscreen / the README live-mode note.
const char* ids[] { ParamID::macro1, ParamID::macro2, ParamID::macro3, ParamID::macro4,
                    ParamID::macro5, ParamID::macro6, ParamID::macro7, ParamID::macro8 };
for (int m = 0; m < 8; ++m)
{
    auto* k = new RotaryKnob (proc.apvts, ids[m], "M" + juce::String (m + 1), proc.getMidiLearn());
    k->setShowValue (false); // no number at rest -> compact
    k->setNameLines (2); // B1: full assignment name (2 lines,
shrink-to-fit)
    k->setTransientReadout (true); // B1: value appears only while adjusting
    k->setDragPixels ((kDragPixelsForFullRange * 10) / 27); // ~2.7x responsive
    k->setBothAxisDrag(); // top-of-window: allow horizontal drag so touch needn't go up into the title
bar
    macros.add (k); addAndMakeVisible (k);
    const int idx = m;
    macroAtt.add (new juce::ParameterAttachment (*proc.apvts.getParameter (ids[m]),
        [this, idx] (float v) { applyMacro (idx, v); }, nullptr));
    // Macro context menu (long-press / right-click): restore the factory default, rename.
    k->addContextMenuItems ("Restore default assignment",
        [this, idx] { proc.restoreMacroDefault (idx); refreshMacroLabels(); if (restoreFocus)
restoreFocus(); });
    k->addContextMenuItems ("Rename macro...", [this, idx] { renameMacro (idx); });
}

master = std::make_unique<RotaryKnob> (proc.apvts, ParamID::masterGain, "MASTER", proc.getMidiLearn());
addAndMakeVisible (*master);

// Voice controls (ex-Global): poly/mono/legato + glide.
mode = std::make_unique<HSelector> (proc.apvts, ParamID::polyMode, proc.getMidiLearn(),
    juce::StringArray { "POLY", "MONO", "LEG" });
glide = std::make_unique<RotaryKnob> (proc.apvts, ParamID::glideTime, "GLIDE", proc.getMidiLearn());
glide->setHelp ("Glide: portamento — the pitch slides between notes over this time (0 = off). Best in mono /
legato.");
analog = std::make_unique<RotaryKnob> (proc.apvts, ParamID::analog, "ANALOG", proc.getMidiLearn()); //
Tier 1b drift
analog->setHelp ("Analog drift: subtle per-voice pitch & pulse-width wander for a livelier, vintage feel");
// #96 Unison: voice-wide stack. UNI count (1 = off), DETune spread, WIDTH (stereo).
uni = std::make_unique<RotaryKnob> (proc.apvts, ParamID::oscUnison, "UNI", proc.getMidiLearn());
uni->setHelp ("Unison: stack this many detuned voices per note (1 = off). The live profile caps high
counts.");
uniDet = std::make_unique<RotaryKnob> (proc.apvts, ParamID::oscUnisonDetune, "DET", proc.getMidiLearn());
uniDet->setHelp ("Unison detune: how far the stacked voices spread in pitch");
uniWid = std::make_unique<RotaryKnob> (proc.apvts, ParamID::oscUnisonWidth, "WID", proc.getMidiLearn());
uniWid->setHelp ("Unison width: how far the stacked voices spread across the stereo field");
// Per-patch TRIM (program level): the patch's own output gain, baked with the sound so a
// patch carries its loudness. Registry-bound -> a LINK target that animates like the rest.
trim = std::make_unique<RotaryKnob> (proc.apvts, ParamID::patchTrim, "TRIM", proc.getMidiLearn());
trim->setHelp ("Patch trim: this patch's own output level (the factory bank is loudness-matched with it).
Saved with the sound.");
addAndMakeVisible (*mode); addAndMakeVisible (*glide); addAndMakeVisible (*analog);
addAndMakeVisible (*uni); addAndMakeVisible (*uniDet); addAndMakeVisible (*uniWid);
addAndMakeVisible (*trim);
// Top-of-window knobs: allow HORIZONTAL drag too (drag right = raise), like the macros
// already do (see the macro loop). A knob pinned near the title bar has no room to drag
// UP to increase, so vertical-only drag left these stuck at the screen edge.
for (auto* k : { master.get(), glide.get(), analog.get(), uni.get(), uniDet.get(), uniWid.get(), trim.get()
})
    k->setBothAxisDrag();

refreshMacroLabels();
startTimerHz (4); // CPU readout + macro-label resync
}

void paint (juce::Graphics& g) override
{
    g.setColour (VASynthLookAndFeel::panelLight());
    g.fillRoundedRectangle (getLocalBounds().toFloat(), 6.0f);

    g.setColour (VASynthLookAndFeel::dim());
    g.setFont (juce::Font (juce::FontOptions (11.0f)));
    g.drawText (statusLine, statusArea, juce::Justification::centredLeft, false);
}

```

```

void resized() override
{
    auto tb = getLocalBounds().reduced (10, 7);

    auto preset = tb.removeFromLeft (326); tb.removeFromLeft (20);    // J4#2: gap before the voice group
    auto pTop = preset.removeFromTop (34);
    presetBtn.setBounds (pTop.removeFromLeft (118).reduced (0, 1));
    save.setBounds (pTop.removeFromLeft (42).reduced (2, 1));
    random.setBounds (pTop.removeFromLeft (62).reduced (2, 1));
    vary.setBounds (pTop.removeFromLeft (46).reduced (2, 1));    // reunited with RANDOM (NEW +
NEIGHBOURHOOD)
    clear.setBounds (pTop.reduced (2, 1));
    preset.removeFromTop (8);    // J4#3: nudge the action row down
    // Global-action row below the preset row: LINK | MOD | OUTPUTS | INPUTS on the right; CPU left.
    auto grow = preset.removeFromTop (26);
    inputs.setBounds (grow.removeFromRight (62).reduced (2, 1));
    outputs.setBounds (grow.removeFromRight (70).reduced (2, 1));
    mod.setBounds (grow.removeFromRight (46).reduced (2, 1));
    link.setBounds (grow.removeFromRight (46).reduced (2, 1));
    statusArea = grow;    // CPU readout fills the space VARY vacated

    auto help_ = tb.removeFromRight (34); tb.removeFromRight (4);
    help.setBounds (help_.reduced (0, 16));
    full.setBounds (tb.removeFromRight (36).reduced (0, 16)); tb.removeFromRight (6);
    master->setBounds (tb.removeFromRight (92)); tb.removeFromRight (4);
    rec.setBounds (tb.removeFromRight (54).reduced (0, 16)); tb.removeFromRight (10);
    panic.setBounds (tb.removeFromRight (62).reduced (0, 16)); tb.removeFromRight (10);    // emergency stop,
left of REC

    // Voice group (poly/mono/legato + glide), just right of the preset area.
    mode->setBounds (tb.removeFromLeft (120).withSizeKeepingCentre (120, 30)); tb.removeFromLeft (6);
    glide->setBounds (tb.removeFromLeft (52)); tb.removeFromLeft (4);
    analog->setBounds (tb.removeFromLeft (52)); tb.removeFromLeft (10);
    // #96 Unison: UNI (count) / DET / WID, just right of ANALOG.
    uni->setBounds (tb.removeFromLeft (46)); tb.removeFromLeft (3);
    uniDet->setBounds (tb.removeFromLeft (46)); tb.removeFromLeft (3);
    uniWid->setBounds (tb.removeFromLeft (46)); tb.removeFromLeft (10);
    trim->setBounds (tb.removeFromLeft (46)); tb.removeFromLeft (10);    // per-patch program level

    // 8 macro knobs, packed to 75% of the middle span (J4#1: closer together), centred.
    const int n = macros.size();
    const int span = tb.getWidth();
    const int mw = juce::jmax (10, (span * 3 / 4) / n);
    tb.removeFromLeft (juce::jmax (0, (span - mw * n) / 2));
    for (int m = 0; m < n; ++m)
        macros[m]->setBounds (tb.removeFromLeft (mw).reduced (2, 0));
}

// Update the shown patch name from outside (e.g. a factory load elsewhere).
void setCurrentName (juce::String n) { currentName = std::move (n); refreshTitle(); }

private:
void timerCallback() override
{
    const int cpu = (int) juce::roundToInt (proc.health.snapshot().cpuPercent);
    auto line = "CPU " + juce::String (juce::jlimit (0, 999, cpu)) + "%";
    if (line != statusLine) { statusLine = line; repaint (statusArea); }
    refreshMacroLabels();    // map can change on Random / preset load

    // A completed knob-tap auto-disarms the source; keep the LINK button label in sync.
    if ((proc.modLinkArmedSource() >= 0) != linkWasArmed)
    { linkWasArmed = proc.modLinkArmedSource() >= 0; refreshLinkButton(); repaintTop(); }

    // Poly/Mono/Legato + glide are per-part VOICE controls; a kit part is always poly
    // and its pads don't glide, so grey them out (disabled) when a kit is the active part.
    const bool kitActive = proc.isPartKit (proc.playFocus());
    if (kitActive != voiceCtrlsDisabled)
    {
        voiceCtrlsDisabled = kitActive;
        if (mode) { mode->setEnabled (! kitActive); mode->setAlpha (kitActive ? 0.35f : 1.0f); }
        if (glide) { glide->setEnabled (! kitActive); glide->setAlpha (kitActive ? 0.35f : 1.0f); }
    }
}

// LINK: armed? disarm. Otherwise pop the source list; picking one arms it and lights every
// destination knob's ring (a top-level repaint) so the next knob tap makes the route.

```

```

void onLinkClicked()
{
    if (proc.modLinkArmedSource() >= 0) { proc.disarmModLink(); refreshLinkButton(); repaintTop(); return; }
    const auto names = ModMatrixPanel::sourceNames(); // index 0 = "None" (skip)
    juce::PopupMenu m;
    for (int s = 1; s < names.size(); ++s) m.addItem (s, names[s]);
    m.showMenuAsync (juce::PopupMenu::Options().withTargetComponent (&link),
        [this] (int r) { if (r > 0) { proc.armModLink (r); refreshLinkButton(); repaintTop(); }
            if (restoreFocus) restoreFocus(); });
}
void refreshLinkButton()
{
    const bool armed = proc.modLinkArmedSource() >= 0;
    link.setColour (juce::TextButton::buttonColourId, armed ? kLinkRing : VASynthLookAndFeel::track());
    link.setColour (juce::TextButton::textColourOffId, armed ? VASynthLookAndFeel::panel() : kLinkRing);
    const auto names = ModMatrixPanel::sourceNames();
    const int s = proc.modLinkArmedSource();
    link.setText (armed && s < names.size() ? names[s].toUpperCase() : juce::String ("LINK"));
}
void repaintTop() { if (auto* t = getTopLevelComponent()) t->repaint(); }

// RANDOM: one press = one shaped-random patch on the focused part (no modes, no toast, no
// long-press picker). The title showing "Random" is the feedback; VARY beside it is the neighbour.
void rollRandom()
{
    juce::Random rng;
    proc.randomizeSound (rng);
    currentName = "Random"; refreshTitle(); refreshMacroLabels();
    if (auto* t = getTopLevelComponent()) t->repaint();
    if (restoreFocus) restoreFocus();
}

void applyMacro (int idx, float value)
{
    auto id = proc.getMacroTargetId (idx);
    if (id.isEmpty()) return;
    // The "focused part level" macro (#55) resolves live to the edit-focused part's level.
    if (id == VASynthProcessor::kFocusLevelTarget)
        id = "part" + juce::String (juce::jlimit (0, 3, proc.editFocus())) + "_level";
    // J1: a SYNC'd LFO ignores its Hz RATE (it follows the stepped note-division), so a macro
    // aimed at that LFO's rate would look dead. Retarget it to the LFO's DIV – a choice param,
    // so setValueNotifyingHost bins the 0..1 macro value onto the nearest division automatically.
    {
        auto on = [this] (const char* pid) { auto* a = proc.apvts.getRawParameterValue (pid); return a &&
a->load() > 0.5f; };
        if (id == ParamID::lfoRate && on (ParamID::lfoSync)) id = ParamID::lfoDiv;
        else if (id == ParamID::lfo2Rate && on (ParamID::lfo2Sync)) id = ParamID::lfo2Div;
        else if (id == ParamID::lfo3Rate && on (ParamID::lfo3Sync)) id = ParamID::lfo3Div;
    }
    if (auto* target = proc.apvts.getParameter (id))
        if (! juce::approximatelyEqual (target->getValue(), value))
            target->setValueNotifyingHost (value);
}
void refreshMacroLabels()
{
    for (int m = 0; m < macros.size(); ++m)
    {
        // Label what the macro CONTROLS: single dest -> its short name; multiple -> a user name
        // if set, else "M3 x2"; none -> the plain "M3" fallback. (H3)
        const int n = proc.macroDestinationCount (m);
        const juce::String fallback = "M" + juce::String (m + 1);
        juce::String label;
        if (n == 0) label = fallback;
        else if (n == 1) label = proc.macroDestinationNames (m).joinIntoString ("");
        else
            { const auto custom = proc.macroCustomName (m);
                label = custom.isEmpty() ? custom : (fallback + " x" + juce::String (n)); }
        macros[m]->setDisplayName (label.isEmpty() ? label : fallback);
    }
}

void renameMacro (int idx)
{
    auto* aw = new juce::AlertWindow ("Rename Macro " + juce::String (idx + 1),
        "A label shown when this macro drives several destinations:",
        juce::MessageBoxIconType::NoIcon, this);
    aw->addTextEditor ("name", proc.macroCustomName (idx));
}

```

```

    aw->addButton ("OK", 1, juce::KeyPress (juce::KeyPress::returnKey));
    aw->addButton ("Cancel", 0, juce::KeyPress (juce::KeyPress::escapeKey));
    aw->enterModalState (true, juce::ModalCallbackFunction::create (
        [this, aw, idx] (int r)
        {
            if (r == 1) { proc.setMacroCustomName (idx, aw->getTextEditorContents ("name").trim());
refreshMacroLabels(); }
            if (restoreFocus) restoreFocus();
        }
    ), true);
}

void refreshTitle() { presetBtn.setButtonText ("synth - " + currentName); }

// Load a patch by name (Init / factory / user) and reflect it in the title.
void loadNamed (const juce::String& n)
{
    if (n == "Init") proc.loadInitPreset();
    else if (proc.factoryPresetLibrary().byName (n) != nullptr) proc.loadFactoryPreset (n);
    else proc.loadUserPreset (n);
    currentName = n; refreshTitle();
}

public:
    // Build the preset Load menu into `m`. Factory categories become collapsible submenus;
    // user presets get a "My Presets" submenu where each patch is itself a Load/Rename/Delete
    // submenu. Pure of UI state (takes hooks) so a test can introspect the tree + fire the actions.
    static void buildPresetMenu (juce::PopupMenu& m,
                                const FactoryPresetLibrary& lib,
                                const PresetManager& presets,
                                std::function<void (const juce::String&)> onLoad,
                                std::function<void (const juce::String&)> onRename,
                                std::function<void (const juce::String&)> onDelete)
    {
        m.addItem ("Init", [onLoad] { onLoad ("Init"); }); // blank slate, pinned at the top

        for (auto& cat : lib.orderedCategories()) // canonical musical order
        {
            juce::PopupMenu sub;
            for (auto& n : lib.namesInCategory (cat)) sub.addItem (n, [onLoad, n] { onLoad (n); });
            m.addSubMenu (cat, sub);
        }

        auto userNames = presets.getPresetNames();
        if (! userNames.isEmpty())
        {
            std::map<juce::String, juce::StringArray> byCat;
            for (auto& n : userNames) byCat[presets.getPresetCategory (n)].add (n);
            juce::StringArray groups = FactoryPresetLibrary::canonicalOrder();
            groups.add ("User");

            juce::PopupMenu mine;
            for (auto& g : groups)
            {
                auto it = byCat.find (g);
                if (it == byCat.end()) continue;
                mine.addSectionHeader (g);
                it->second.sort (true);
                for (auto& n : it->second)
                {
                    juce::PopupMenu actions;
                    actions.addItem ("Load", [onLoad, n] { onLoad (n); });
                    actions.addItem ("Rename...", [onRename, n] { onRename (n); });
                    actions.addItem ("Delete", [onDelete, n] { onDelete (n); });
                    mine.addSubMenu (n, actions);
                }
            }
            m.addSeparator();
            m.addSubMenu ("My Presets", mine);
        }
    }

private:
    void showPresetMenu()
    {
        juce::PopupMenu m;
        buildPresetMenu (m, proc.factoryPresetLibrary(), presets,

```

```

        [this] (const juce::String& n) { loadNamed (n); },
        [this] (const juce::String& n) { showRenameDialog (n); },
        [this] (const juce::String& n) { confirmDelete (n); });
    m.showMenuAsync (juce::PopupMenu::Options().withTargetComponent (presetBtn),
        [this] (int) { if (restoreFocus) restoreFocus(); });
}

void showRenameDialog (const juce::String& oldName)
{
    auto* aw = new juce::AlertWindow ("Rename Preset", "New name for \"" + oldName + "\":",
        juce::MessageBoxIconType::NoIcon, this);
    aw->addTextEditor ("name", oldName);
    aw->addButton ("Rename", 1, juce::KeyPress (juce::KeyPress::returnKey));
    aw->addButton ("Cancel", 0, juce::KeyPress (juce::KeyPress::escapeKey));
    aw->enterModalState (true, juce::ModalCallbackFunction::create (
        [this, aw, oldName] (int result)
        {
            if (result == 1)
            {
                const auto n = aw->getTextEditorContents ("name").trim();
                if (presets.rename (oldName, n) && currentName == oldName)
                { currentName = juce::File::createLegalFileName (n).trim(); refreshTitle(); }
            }
            if (restoreFocus) restoreFocus();
        })), true);
}

void confirmDelete (const juce::String& name)
{
    auto* aw = new juce::AlertWindow ("Delete Preset", "Delete \"" + name + "\"? This cannot be undone.",
        juce::MessageBoxIconType::WarningIcon, this);
    aw->addButton ("Delete", 1, juce::KeyPress (juce::KeyPress::returnKey));
    aw->addButton ("Cancel", 0, juce::KeyPress (juce::KeyPress::escapeKey));
    aw->enterModalState (true, juce::ModalCallbackFunction::create (
        [this, aw, name] (int result)
        {
            juce::ignoreUnused (aw);
            if (result == 1 && presets.remove (name) && currentName == name)
            { currentName = "Init"; refreshTitle(); }
            if (restoreFocus) restoreFocus();
        })), true);
}

void showSaveDialog()
{
    auto* aw = new juce::AlertWindow ("Save Preset", "Preset name:",
        juce::MessageBoxIconType::NoIcon, this);
    aw->addTextEditor ("name", "");
    juce::StringArray cats = FactoryPresetLibrary::canonicalOrder();
    cats.add ("User");
    aw->addComboBox ("cat", cats, "Category:");
    if (auto* cb = aw->getComboBoxComponent ("cat")) cb->setText ("User", juce::dontSendNotification);
    aw->addButton ("Save", 1, juce::KeyPress (juce::KeyPress::returnKey));
    aw->addButton ("Cancel", 0, juce::KeyPress (juce::KeyPress::escapeKey));
    aw->enterModalState (true, juce::ModalCallbackFunction::create (
        [this, aw] (int result)
        {
            if (result == 1)
            {
                const auto n = aw->getTextEditorContents ("name");
                auto* cb = aw->getComboBoxComponent ("cat");
                const auto cat = cb != nullptr ? cb->getText() : juce::String ("User");
                if (presets.save (n, cat)) { currentName = n; refreshTitle(); }
            }
            if (restoreFocus) restoreFocus();
        })), true);
}

VASynthProcessor& proc;
PresetManager& presets;
std::function<void()> restoreFocus, toggleHelp, toggleFullscreen;

juce::TextButton presetBtn, save, random, clear, rec, full, help;
juce::TextButton vary; // H5: perturb the current patch
juce::TextButton panic; // all-notes-off (live + loops + seq/arp + samples)
juce::TextButton link, mod, inputs, outputs; // global-action row

```

```

bool linkWasArmed = false;
inline static const juce::Colour kLinkRing { 0xff4bb3c4 }; // LINK cyan (matches the knob armed ring)
juce::OwnedArray<RotaryKnob> macros;
juce::OwnedArray<juce::ParameterAttachment> macroAtt;
std::unique_ptr<RotaryKnob> master, glide, analog, uni, uniDet, uniWid, trim;
std::unique_ptr<HSelector> mode;
bool voiceCtrlsDisabled = false; // mode/glide greyed while a kit part is active
juce::Rectangle<int> statusArea;
juce::String statusLine { "CPU 0%" }, currentName { "Init" };

JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (TopBar)
};

```

===== FILE 72/75: Source/UI/UiText.h (25 lines) =====

// Synth — Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.

#pragma once

#include <juce_core/juce_core.h>

```

// =====
// UTF-8 string convention (see the mojibake defect, #56 follow-up).
//
// CONVENTION: juce::String's `const char*` constructor decodes as ASCII/Latin-1,
// NOT UTF-8 (juce_String.cpp: `CharPointer_ASCII`). So a literal like
// "\xe2\x80\x94" (an em-dash in UTF-8) becomes THREE garbage codepoints
// (0xE2 -> "â", ...). In a Debug build a jassert fires; in RELEASE it is
// stripped, so the mojibake ships silently – exactly how the MOD overlay's
// arrows/dashes turned into "â".
//
// RULE: any string literal containing a byte > 0x7f MUST be wrapped in u8(),
// which routes it through CharPointer_UTF8 and decodes correctly. Prefer
// DRAWING decorative glyphs (arrows, the remove X) as juce::Path instead of
// text – no font-glyph dependency and no encoding hazard at all. Reserve u8()
// for genuine text (a real em-dash label, degree/times signs, accented names).
// grep for `x` escapes in Source/UI to audit.
// =====
namespace uitext
{
    inline juce::String u8 (const char* utf8Literal) { return juce::String::fromUTF8 (utf8Literal); }
}

```

===== FILE 73/75: Source/UI/VASynthLookAndFeel.h (192 lines) =====

// Synth — Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.

#pragma once

#include <juce_gui_basics/juce_gui_basics.h>

```

// =====
// One dark hardware-panel LookAndFeel for the whole UI – no per-widget style
// hacks. High-contrast fader tracks/thumbs, subtle section tints, legible value
// text; readable at arm's length on a touchscreen. Fader thumbs are sized for
// fingers, not mouse pointers.
// =====

```

```

class VASynthLookAndFeel : public juce::LookAndFeel_V4
{
public:
    // Palette.
    static juce::Colour panel() { return juce::Colour (0xff1c1f24); }
    static juce::Colour panelLight() { return juce::Colour (0xff262a31); }
    static juce::Colour ink() { return juce::Colour (0xffe8eaed); }
    static juce::Colour dim() { return juce::Colour (0xff9aa0a8); }
    static juce::Colour accent() { return juce::Colour (0xff46c9b0); } // teal
    static juce::Colour accentWarm() { return juce::Colour (0xffff0a04b); } // amber (learn)
    // Per-LFO identity colour, used everywhere an LFO appears (panel header, ring, mod-anim arc,
    // MOD-overlay links list): LFO 1 amber, LFO 2 teal, LFO 3 violet. So a colour means "LFO N".
    static juce::Colour lfoColour (int idx)
    {
        switch (idx) { case 1: return juce::Colour (0xff46c9b0); // teal (greenish-blue)
                      case 2: return juce::Colour (0xffb07cff); // violet (purple)
                      default: return juce::Colour (0xffff0a04b); } // amber (LFO 1 / default)
    }
    static juce::Colour track() { return juce::Colour (0xff10131a); }

    VASynthLookAndFeel()
    {
        setColour (juce::ResizableWindow::backgroundColourId, panel());
        setColour (juce::Slider::textBoxTextColourId, ink());
    }
}

```

```

        setColour (juce::Slider::textBoxOutlineColourId, juce::Colours::transparentBlack);
        setColour (juce::Label::textColourId, ink());
        setColour (juce::ComboBox::backgroundColourId, panellLight());
        setColour (juce::ComboBox::textColourId, ink());
        setColour (juce::ComboBox::outlineColourId, juce::Colour (0xff3a3f47));
        setColour (juce::PopupMenu::backgroundColourId, panellLight());
        setColour (juce::PopupMenu::textColourId, ink());
    }

    juce::Font sectionFont() { return juce::Font (juce::FontOptions (13.0f, juce::Font::bold)); }
    juce::Font labelFont()   { return juce::Font (juce::FontOptions (12.0f)); }
    juce::Font valueFont()   { return juce::Font (juce::FontOptions (juce::Font::getDefaultMonospacedFontName(),
12.0f, juce::Font::plain)); }

    // Vertical fader: recessed track, bright filled portion, chunky finger thumb.
    void drawLinearSlider (juce::Graphics& g, int x, int y, int width, int height,
                          float sliderPos, float minPos, float maxPos,
                          const juce::Slider::SliderStyle style, juce::Slider& s) override
    {
        if (style != juce::Slider::LinearVertical)
        {
            juce::LookAndFeel_V4::drawLinearSlider (g, x, y, width, height, sliderPos, minPos, maxPos, style, s);
            return;
        }

        const float cx = x + width * 0.5f;
        const float trackW = 8.0f;
        juce::Rectangle<float> trackR (cx - trackW * 0.5f, (float) y + 4.0f, trackW, (float) height - 8.0f);

        g.setColour (track());
        g.fillRoundedRectangle (trackR, 4.0f);

        // filled portion from the bottom up to the thumb
        auto filled = trackR;
        filled.setTop (juce::jlimit (trackR.getTop(), trackR.getBottom(), sliderPos));
        g.setColour (accent().withAlpha (s.isEnabled() ? 0.95f : 0.4f));
        g.fillRoundedRectangle (filled, 4.0f);

        // thumb – big finger target: >= 56 px wide at default scale.
        const float thumbW = juce::jmin ((float) width - 2.0f, 56.0f);
        const float thumbH = 20.0f;
        juce::Rectangle<float> thumb (cx - thumbW * 0.5f, sliderPos - thumbH * 0.5f, thumbW, thumbH);
        g.setColour (juce::Colour (0xffdfe3e8));
        g.fillRoundedRectangle (thumb, 4.0f);
        g.setColour (juce::Colours::black.withAlpha (0.35f));
        g.drawRoundedRectangle (thumb, 4.0f, 1.0f);
        g.setColour (accent().darker (0.2f));
        g.fillRect (thumb.reduced (5.0f, 0.0f).withHeight (2.0f).withY (thumb.getCentreY() - 1.0f));
    }

    // Dark rotary knob: recessed track arc, accent value arc, raised body with a
    // pointer. Sized to read on a touchscreen; the whole widget is the finger target.
    void drawRotarySlider (juce::Graphics& g, int x, int y, int width, int height,
                          float sliderPos, float startAngle, float endAngle,
                          juce::Slider& s) override
    {
        auto bounds = juce::Rectangle<int> (x, y, width, height).toFloat().reduced (3.0f);
        const float radius = juce::jmin (bounds.getWidth(), bounds.getHeight()) * 0.5f;
        const float cx = bounds.getCentreX(), cy = bounds.getCentreY();
        const float angle = startAngle + sliderPos * (endAngle - startAngle);
        const float lineW = juce::jmax (2.5f, radius * 0.16f);
        const float arcR = radius - lineW * 0.5f;

        juce::Path track;
        track.addCentredArc (cx, cy, arcR, arcR, 0.0f, startAngle, endAngle, true);
        g.setColour (VASynthLookAndFeel::track());
        g.strokePath (track, juce::PathStrokeType (lineW, juce::PathStrokeType::curved,
juce::PathStrokeType::rounded));

        juce::Path val;
        val.addCentredArc (cx, cy, arcR, arcR, 0.0f, startAngle, angle, true);
        g.setColour (accent().withAlpha (s.isEnabled() ? 0.95f : 0.4f));
        g.strokePath (val, juce::PathStrokeType (lineW, juce::PathStrokeType::curved,
juce::PathStrokeType::rounded));

        const float bodyR = radius - lineW * 1.7f;

```

```

        g.setColour (panelLight().brighter (0.05f));
        g.fillEllipse (cx - bodyR, cy - bodyR, bodyR * 2.0f, bodyR * 2.0f);
        g.setColour (juce::Colour (0xff3a3f47));
        g.drawEllipse (cx - bodyR, cy - bodyR, bodyR * 2.0f, bodyR * 2.0f, 1.0f);

        const juce::Point<float> tip (cx + std::cos (angle - juce::MathConstants<float>::halfPi) * bodyR * 0.85f,
                                     cy + std::sin (angle - juce::MathConstants<float>::halfPi) * bodyR * 0.85f);
        g.setColour (ink());
        g.drawLine (juce::Line<float> (juce::Point<float> (cx, cy), tip), juce::jmax (2.0f, radius * 0.10f));
    }

    // Fit button text to the button width (shrink the font rather than truncate),
    // so no label is ever clipped at any size.
    void drawButtonText (juce::Graphics& g, juce::TextButton& b, bool, bool) override
    {
        auto area = b.getLocalBounds().reduced (5, 2);
        auto text = b.getButtonText();
        auto font = getTextButtonFont (b, b.getHeight());
        const float w = juce::GlyphArrangement::getStringWidth (font, text);
        if (w > (float) area.getWidth() && w > 0.0f)
            font.setHeight (juce::jmax (8.0f, font.getHeight() * (float) area.getWidth() / w));
        g.setFont (font);
        g.setColour (b.findColour (b.getToggleState() ? juce::TextButton::textColourOnId
                                           : juce::TextButton::textColourOffId)
                    .withMultipliedAlpha (b.isEnabled() ? 1.0f : 0.5f));
        g.drawText (text, area, juce::Justification::centred, false);
    }

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (VASynthLookAndFeel)
};

// -----
// A titled panel section with a subtle tint and outline. Children are laid out
// by the owner; the section just draws the frame + header.
class Section : public juce::Component
{
public:
    Section (juce::String titleText, juce::Colour tintColour)
        : title (std::move (titleText)), tint (tintColour) {}

    void paint (juce::Graphics& g) override
    {
        auto r = getLocalBounds().toFloat().reduced (2.0f);
        g.setColour (VASynthLookAndFeel::panelLight().interpolatedWith (tint, 0.10f));
        g.fillRoundedRectangle (r, 7.0f);
        g.setColour (tint.withAlpha (0.55f));
        g.drawRoundedRectangle (r, 7.0f, 1.2f);

        g.setColour (VASynthLookAndFeel::ink());
        g.setFont (juce::Font (juce::FontOptions (15.0f, juce::Font::bold)));
        g.drawText (title.toUpperCase(), getLocalBounds().removeFromTop (headerHeight).withTrimmedLeft (10),
                   juce::Justification::centredLeft, false);
    }

    // Content area below the header (owner lays out children here).
    juce::Rectangle<int> contentBounds() const
    {
        return getLocalBounds().withTrimmedTop (headerHeight).reduced (8, 6);
    }

    // Lay out child controls left to right; each control's "layoutFlex" property
    // sets its relative width (faders 1.0, segmented controls wider).
    void resized() override
    {
        auto kids = getChildren();
        if (kids.isEmpty()) return;
        juce::FlexBox fb;
        fb.flexDirection = juce::FlexBox::Direction::row;
        for (auto* c : kids)
        {
            const float flex = (float) (double) c->getProperties().getWithDefault ("layoutFlex", 1.0);
            fb.items.add (juce::FlexItem (*c).withFlex (flex).withMargin (3.0f));
        }
        fb.performLayout (contentBounds());
    }

```

```

static constexpr int headerHeight = 24;

private:
    juce::String title;
    juce::Colour tint;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (Section)
};

===== FILE 74/75: Source/UI/Widgets.h (1176 lines) =====
// Synth — Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.
#pragma once
#include <juce_audio_processors/juce_audio_processors.h>
#include "VASynthLookAndFeel.h"
#include "ModLink.h"
#include "ModAnim.h"
#include "../MidiLearnManager.h"

// =====
// Touch-first, MIDI-learnable UI widgets bound to APVTS parameters. The APVTS
// attachment classes keep GUI <-> APVTS <-> DAW automation <-> MIDI-learn all in
// sync with no custom glue. Every widget refuses keyboard focus so the QWERTY
// computer-keyboard note input keeps working while twisting controls.
//
// MIDI-learn: right-click (mouse) or long-press (touch) any control -> arms
// MidiLearnManager for it (pulsing amber outline); the next incoming CC binds it
// and a small "CCnn" badge appears. The same gesture offers clear-mapping.
// =====

// R2 touch: pixels of finger travel to move a fader/knob across its FULL range
// (JUCE's Slider default is 250). Higher = less sensitive, so small movements don't
// over-shoot values during live play. Applied consistently to every fader + knob.
// ONE knob to re-tune after the layout rebuild changes control sizes (larger controls
// want more travel). ~313 = JUCE's 250 x 1.25 (~25% more travel / ~20% less per pixel).
inline constexpr int kDragPixelsForFullRange = 313;

// Clean, arm's-length-legible value readout for a float parameter: sensible
// decimal places for the magnitude, plus the parameter's own unit label (Hz, s,
// ms, ct) — JUCE's getCurrentValueAsText() shows neither (it dumps full float
// precision like "299.9999695" and never appends the label). Purely cosmetic;
// the parameter value/state is untouched.
inline juce::String formatParamValue (juce::RangedAudioParameter* p)
{
    if (auto* fp = dynamic_cast<juce::AudioParameterFloat*> (p))
    {
        const float v = fp->get();
        const float a = std::abs (v);
        juce::String num = a >= 100.0f ? juce::String (juce::roundToInt (v))
            : a >= 10.0f ? juce::String (v, 1)
            : a >= 1.0f ? juce::String (v, 2)
            : juce::String (v, 3);
        const juce::String unit = fp->getLabel();
        return unit.isEmpty() ? num : num + " " + unit;
    }
    return p != nullptr ? p->getCurrentValueAsText() : juce::String();
}

// Shared MIDI-learn interaction for a control bound to one parameter.
class LearnableComponent : public juce::Component,
    public juce::SettableTooltipClient,
    private juce::Timer
{
public:
    LearnableComponent (MidiLearnManager& learnToUse, juce::String paramIdToUse)
        : learn (learnToUse), paramID (std::move (paramIdToUse))
    {
        setWantsKeyboardFocus (false);
    }

    // Use the parameter's full registered name (e.g. "Filter Cutoff") as the hover tooltip.
    // `inner` is the interactive child that actually receives the mouse (a slider) — it needs its
    // own tooltip since the TooltipWindow reads the leaf under the cursor, not this parent.
    void setTooltipFromParam (juce::AudioProcessorValueTreeState& apvts, juce::Component* inner = nullptr)
    {
        if (auto* p = apvts.getParameter (paramID))
        {

```

```

        setTooltip (p->getName (128));
        if (auto* ttc = dynamic_cast<juce::SettableTooltipClient*> (inner)) ttc->setTooltip (getTooltip());
    }
}

// Override the hover tooltip with a self-explanatory description (for cryptic controls:
// phase RS/RN/FR, ANALOG, DRIVE, LFO SYNC/DIV, ...). `inner` is the leaf the TooltipWindow
// reads under the cursor – pass the interactive child (a slider) so it carries the text too.
void setHelp (const juce::String& text, juce::Component* inner = nullptr)
{
    setTooltip (text);
    if (auto* ttc = dynamic_cast<juce::SettableTooltipClient*> (inner)) ttc->setTooltip (text);
}

// Call from a subclass ctor after the inner control is added, passing it so
// long-press/right-click on the control is captured here too.
void listenForLearnGestures (juce::Component& inner)
{
    inner.addMouseListener (this, true);
}

void mouseDown (const juce::MouseEvent& e) override
{
    // LINK routing takes priority (whole cell): if a source is armed and this control is a
    // registry target, a tap connects it. Guard eventComponent==this so the inner-slider
    // listener path (which also lands here) doesn't double-fire – the ModSlider handles taps
    // that land on the slider itself.
    if (e.eventComponent == this && beginLinkGesture (e)) return;
    if (e.mods.isPopupMenu()) { showLearnMenu(); return; }
    // #139: tapping a control that is already armed for learn CANCELS it (escape hatch for
    // an accidental long-press that left the amber highlight stuck on).
    if (learn.isLearningParam (paramID)) { learn.armLearn (juce::String()); stopTimer(); repaint(); return; }
    pressStart = juce::Time::getMillisecondCounter();
    longPressArmed = true;
    startTimer (60); // poll for the long-press threshold
}

void mouseDrag (const juce::MouseEvent& e) override
{
    if (e.eventComponent == this && linkDepthActive) { dragLinkDepth (e); return; }
    if (e.getDistanceFromDragStart() > 8) longPressArmed = false; // it's a value drag
}

void mouseUp (const juce::MouseEvent& e) override
{
    if (e.eventComponent == this && linkDepthActive) { endLinkGesture(); return; }
    // #148: a quick tap (released before the long-press fired) on the armed LFO DEST cancels
    // arm mode – the "short-press ON cancels" escape hatch.
    if (longPressArmed && ! e.mouseWasDraggedSinceMouseDown())
        && onShortPressOverride && onShortPressOverride()) { longPressArmed = false; stopTimer(); repaint();
return; }
    longPressArmed = false;
}

// ---- Mod-matrix TARGET support (the SINGLE shared path – #56 follow-up). Wired centrally
// from the destination registry (see the editor), never per control. Any control whose
// parameter is in the registry gets the connect-ring, whole-cell tap-to-connect + 2 s
// depth-drag, and a live animation indicator; everything else gets none of it. ----
void setModTarget (ModLinkController& c, int dest, std::function<float()> animFn)
{ modLink = &c; modDest = dest; modAnimFn = std::move (animFn); modTargetAttached(); }
bool isModTarget() const { return modLink != nullptr && modDest > 0; }
bool isLinkArmable() const { return isModTarget() && modLink->linkArmed(); }
float modAnim() const { return modAnimFn ? modAnimFn() : 0.0f; } // normalized offset for the indicator
// Per-LFO colour + armed context for the mod-anim indicator (#148). lfoIdx: which LFO (0..2)
// colours this dest, or -1. armedLfo: the LFO armed for link, or -1 (arm-mode "editable picture").
int modLfoIdx() const { return modLink ? modLink->lfoDrivingDest (modDest) : -1; }
int modArmedLfo() const { return modLink ? modLink->armedLfo() : -1; }
juce::Colour modColour() const
{ const int i = modLfoIdx(); return i >= 0 ? VASynthLookAndFeel::lfoColour (i) :
VASynthLookAndFeel::accentWarm(); }
bool inLinkDrag() const { return linkDepthActive; }
// #148 LFO Link: the LFO DEST selector overrides its long-press (arm/commit) and a quick tap
// while armed (cancel) – instead of MIDI-learn / value-cycle. Each returns true if it handled it.
std::function<bool()> onLongPressOverride, onShortPressOverride;

bool beginLinkGesture (const juce::MouseEvent& e)
{
    if (modLink == nullptr || modDest <= 0) return false;

```

```

const juce::uint32 now = juce::Time::getMillisecondCounter();
if (modLink->linkArmed())
{
    const int slot = modLink->completeModLink (modDest);
    if (slot < 0) return false; // matrix full -> behave as a normal control
    linkSlot = slot; linkDepthActive = true; linkDownY = e.getPosition().y;
    linkDownDepth = modLink->modRouteDepth (slot); linkTime = now; repaint(); return true;
}
if (linkSlot >= 0 && now - linkTime < 2000) // re-grab the just-made route within ~2 s
{ linkDepthActive = true; linkDownY = e.getPosition().y; linkDownDepth = modLink->modRouteDepth (linkSlot);
return true; }
return false;
}
void dragLinkDepth (const juce::MouseEvent& e)
{
    const float d = juce::jlimit (-1.0f, 1.0f, linkDownDepth + (float) (linkDownY - e.getPosition().y) *
0.007f);
    modLink->setModRouteDepth (linkSlot, d); repaint();
}
void endLinkGesture() { linkDepthActive = false; linkTime = juce::Time::getMillisecondCounter(); }

// #148 Inc 3 – slide-to-bounds. In sticky LFO-Link mode a press+drag on a CONTINUOUS target
// sweeps the knob to define its modulation bounds [lo,hi]; release parks the value at the
// midpoint and creates the route at the signed half-range depth. A pure tap = full-scale route.
bool armedForBounds() const { return modLink != nullptr && modDest > 0 && modLink->armedLfo() >= 0; }
bool beginBoundsCapture (float norm, bool continuous)
{
    if (! armedForBounds() || ! continuous) return false;
    boundsCapturing = true; boundsStart = boundsMin = boundsMax = juce::jlimit (0.0f, 1.0f, norm); return true;
}
bool inBoundsCapture() const { return boundsCapturing; }
void updateBounds (float norm)
{ norm = juce::jlimit (0.0f, 1.0f, norm); boundsMin = std::min (boundsMin, norm); boundsMax = std::max
(boundsMax, norm); repaint(); }
// Ends the capture. Returns the midpoint (0..1) to park the knob at, or -1 for a tap (full-scale).
float endBoundsCapture (bool dragged, float releaseNorm)
{
    boundsCapturing = false; repaint();
    if (! dragged || boundsMax - boundsMin < 0.01f) { if (modLink) modLink->completeModLink (modDest); return
-1.0f; }
    const float mid = 0.5f * (boundsMin + boundsMax), half = 0.5f * (boundsMax - boundsMin);
    const float sign = (juce::jlimit (0.0f, 1.0f, releaseNorm) >= boundsStart) ? 1.0f : -1.0f;
    if (modLink) modLink->setModLinkBounds (modDest, sign * half);
    return mid;
}
float boundsLo() const { return boundsMin; }
float boundsHi() const { return boundsMax; }

// Cyan connect-ring – every armable target draws it (call from paint()).
void paintModRing (juce::Graphics& g)
{
    if (! isLinkArmable()) return;
    g.setColour (juce::Colour (0xff4bb3c4).withAlpha (0.9f));
    g.drawRoundedRectangle (getLocalBounds().toFloat().reduced (1.5f), 5.0f, 2.0f);
}

// Owner-supplied extra right-click/long-press items (e.g. a macro's Restore-default / Rename).
void addContextMenuItems (juce::String label, std::function<void()> action)
{ extraMenuItems.emplace_back (std::move (label), std::move (action)); }

// Subclasses call this in paint() to draw the armed outline + CC badge.
void paintLearnDecorations (juce::Graphics& g)
{
    const int cc = learn.getCCForParam (paramID);
    if (cc >= 0)
    {
        // A compact CC chip. Placed top-right by default; for controls whose name needs the
        // full top (two-line macros) it drops to the bottom-right so it never collides.
        const int bw = juce::jmin (getWidth() - 2, ccBadgeAtBottom ? 30 : 34);
        auto strip = ccBadgeAtBottom ? getLocalBounds().removeFromBottom (12) : getLocalBounds().removeFromTop
(12);
        auto badge = strip.removeFromRight (bw).toFloat().reduced (0.5f);
        g.setColour (VASynthLookAndFeel::accent().withAlpha (0.92f));
        g.fillRectRoundedRectangle (badge, 2.5f);
        g.setColour (juce::Colours::black);
        g.setFont (juce::Font (juce::FontOptions (9.0f, juce::Font::bold)));
    }
}

```

```

        g.drawText ("CC" + juce::String (cc), badge, juce::Justification::centred, false);
    }
    if (learn.isLearningParam (paramID))
    {
        const float a = 0.35f + 0.45f * (float) std::abs (std::sin (juce::Time::getMillisecondCounter() *
0.006));
        g.setColour (VASynthLookAndFeel::accentWarm().withAlpha (a));
        g.drawRoundedRectangle (getLocalBounds().toFloat().reduced (1.5f, 5.0f, 2.5f);
    }
}

const juce::String& parameterID() const { return paramID; }

// Double-click numeric entry (R3 Group 4): a transient TextEditor over the control,
// parsed via the parameter's own text<->value conversion. QWERTY note input is
// auto-suppressed while a TextEditor holds focus (the editor watchdog) and reclaimed
// when it closes. No-op unless a subclass called enableNumericEntry().
void mouseDoubleClick (const juce::MouseEvent&) override
{
    if (numParam == nullptr || numEditor != nullptr) return;
    longPressArmed = false;
    numEditor = std::make_unique<juce::TextEditor>();
    numEditor->setJustification (juce::Justification::centred);
    numEditor->setText (numParam->getCurrentValueAsText(), false);
    numEditor->setBounds (getLocalBounds().withSizeKeepingCentre (juce::jmin (getWidth(), 76), 20));
    numEditor->onReturnKey = [this] { commitNumericEntry(); };
    numEditor->onEscapeKey = [this] { closeNumericEntry(); };
    numEditor->onFocusLost = [this] { commitNumericEntry(); };
    addAndMakeVisible (*numEditor);
    numEditor->selectAll();
    numEditor->grabKeyboardFocus();
}

protected:
    void enableNumericEntry (juce::RangedAudioParameter* p) { numParam = p; }
    virtual void modTargetAttached() {} // subclass creates its animation indicator (knob arc / fader ghost)
public:
    // True once this control has actually built its motion indicator. A mod TARGET (isModTarget)
    // whose hasModIndicator() is false is wired-but-dead (data flows, nothing draws – the #12 NOISE
    // bug). Tests assert every target has one, catching the missing-override class of defect.
    virtual bool hasModIndicator() const { return false; }
protected:
    bool ccBadgeAtBottom = false; // move the CC chip to the bottom (two-line-name controls)

    // Mod-target state (shared by every registry control). Protected so ModSlider + subclass
    // indicators can read what they need through the public accessors above.
    ModLinkController* modLink = nullptr;
    int modDest = 0, linkSlot = -1, linkDownY = 0;
    bool linkDepthActive = false;
    float linkDownDepth = 0.0f;
    juce::uint32 linkTime = 0;
    bool boundsCapturing = false; // #148 Inc 3: slide-to-bounds capture in progress
    float boundsStart = 0.0f, boundsMin = 0.0f, boundsMax = 0.0f;
    juce::uint32 armStart = 0; // #139: when learn was armed, for the stuck-highlight auto-timeout
    std::function<float()> modAnimFn;
    std::vector<std::pair<juce::String, std::function<void()>>> extraMenuItems;

private:
    void commitNumericEntry()
    {
        if (numEditor == nullptr) return;
        if (numParam != nullptr)
            numParam->setValueNotifyingHost (juce::jlimit (0.0f, 1.0f, numParam->getValueForText
(numEditor->getText())));
        closeNumericEntry();
    }
    void closeNumericEntry()
    {
        if (numEditor == nullptr) return;
        numEditor->onReturnKey = nullptr; numEditor->onEscapeKey = nullptr; numEditor->onFocusLost = nullptr;
        std::shared_ptr<juce::TextEditor> dead (numEditor.release()); // defer the delete so we
        dead->setVisible (false); // never free it inside its
        juce::MessageManager::callAsync ([dead] {}); // own callback (UAF-safe)
        repaint();
    }
}

```

```

void timerCallback() override
{
    if (longPressArmed && juce::Time::getMillisecondCounter() - pressStart > 500)
    {
        longPressArmed = false;
        if (onLongPressOverride && onLongPressOverride()) { stopTimer(); repaint(); return; } // #148: LFO
    }
    dest arms link, not learn
    armLearn();
}
if (learn.isLearningParam (paramID))
{
    // #139: an accidental stationary long-press arms learn (amber pulse); it would
    // otherwise stay armed forever if no CC ever arrives, leaving the highlight stuck.
    // Auto-disarm after a grace period so it always clears itself.
    if (juce::Time::getMillisecondCounter() - armStart > kLearnArmTimeoutMs)
    { learn.armLearn (juce::String()); stopTimer(); repaint(); return; }
    repaint(); // pulse while armed
}
else if (! longPressArmed) stopTimer();
}

static constexpr juce::uint32 kLearnArmTimeoutMs = 8000; // #139: auto-clear a stuck learn-arm

void armLearn()
{
    learn.armLearn (paramID);
    armStart = juce::Time::getMillisecondCounter();
    startTimer (33); // ~30 Hz pulse while armed
    repaint();
}

void showLearnMenu()
{
    juce::PopupMenu m;
    m.addItem (1, "MIDI-learn this control");
    const int cc = learn.getCCForParam (paramID);
    if (cc >= 0) m.addItem (2, "Clear mapping (CC" + juce::String (cc) + ")");
    if (! extraMenuItems.empty())
    {
        m.addSeparator();
        int id = 100;
        for (auto& it : extraMenuItems) m.addItem (id++, it.first);
    }
    m.showMenuAsync (juce::PopupMenu::Options().withTargetComponent (this),
                    [this] (int r)
                    {
                        if (r == 1) armLearn();
                        else if (r == 2) { learn.clearParam (paramID); repaint(); }
                        else if (r >= 100 && r - 100 < (int) extraMenuItems.size())
    extraMenuItems[(std::size_t) (r - 100)].second();
                    });
}

MidiLearnManager& learn;
juce::String paramID;
juce::uint32 pressStart = 0;
bool longPressArmed = false;
juce::RangedAudioParameter* numParam = nullptr; // double-click numeric entry target
std::unique_ptr<juce::TextEditor> numEditor;
};

// A slider that defers to its owning LearnableComponent for the LINK gesture: a tap on the
// slider connects the armed source (and drives the 2 s depth-drag) instead of moving the value.
// Shared by RotaryKnob and LabelledFader so BOTH are mod targets through the one path.
struct ModSlider : juce::Slider
{
    explicit ModSlider (LearnableComponent& o) : owner (o) {}
    bool continuousRange() const { return getNormalisableRange().interval <= 0.0; } // stepped -> tap-only (no
bounds drag)
    void mouseDown (const juce::MouseEvent& e) override
    {
        // #148 Inc 3: sticky LFO-Link mode -> drag the knob to set its modulation bounds (continuous only).
        if (owner.beginBoundsCapture ((float) valueToProportionOfLength (getValue()), continuousRange()))
        { juce::Slider::mouseDown (e); return; } // let the knob follow the
pointer
        if (! owner.beginLinkGesture (e)) juce::Slider::mouseDown (e);
    }
};

```

```

    }
    void mouseDrag (const juce::MouseEvent& e) override
    {
        if (owner.inBoundsCapture()) { juce::Slider::mouseDrag (e); owner.updateBounds ((float)
valueToProportionOfLength (getValue())); return; }
        if (owner.inLinkDrag()) owner.dragLinkDepth (e); else juce::Slider::mouseDrag (e);
    }
    void mouseUp (const juce::MouseEvent& e) override
    {
        if (owner.inBoundsCapture())
        {
            juce::Slider::mouseUp (e);
            const float mid = owner.endBoundsCapture (e.mouseWasDraggedSinceMouseDown(), (float)
valueToProportionOfLength (getValue()));
            if (mid >= 0.0f) setValue (proportionOfLengthToValue (mid), juce::sendNotificationSync); // park at
the midpoint
            return;
        }
        if (owner.inLinkDrag()) owner.endLinkGesture(); else juce::Slider::mouseUp (e);
    }
    LearnableComponent& owner;
};

// -----
// Hardware-style on/off kill switch bound to a bool parameter (lit = on).
class PowerToggle : public juce::Component
{
public:
    PowerToggle (juce::AudioProcessorValueTreeState& apvts, const juce::String& pid, juce::String label)
        : name (std::move (label)), paramID (pid)
    {
        btn.setClickingTogglesState (true);
        btn.setWantsKeyboardFocus (false);
        btn.setButtonText (name);
        btn.setColour (juce::TextButton::buttonColourId, VASynthLookAndFeel::track());
        btn.setColour (juce::TextButton::buttonOnColourId, VASynthLookAndFeel::accent());
        btn.setColour (juce::TextButton::textColourOnId, juce::Colours::black);
        btn.setColour (juce::TextButton::textColourOffId, VASynthLookAndFeel::dim());
        addAndMakeVisible (btn);
        attachment = std::make_unique<juce::ButtonParameterAttachment> (*apvts.getParameter (pid), btn);
        if (auto* p = apvts.getParameter (pid)) btn.setTooltip (p->getName (128)); // hover -> full name
        getProperties().set ("layoutFlex", 0.55);
    }

    void setHelp (const juce::String& text) { btn.setTooltip (text); } // custom hover help
    const juce::String& parameterID() const { return paramID; } // for the section guide's marker lookup
    void resized() override { btn.setBounds (getLocalBounds().reduced (2)); }

private:
    juce::String name;
    juce::String paramID;
    juce::TextButton btn;
    std::unique_ptr<juce::ButtonParameterAttachment> attachment;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (PowerToggle)
};

// -----
// Vertical fader + name + live value readout, bound to a float parameter.
class LabelledFader : public LearnableComponent
{
public:
    // emphasis = a prominent fader (e.g. MASTER): larger, higher-contrast label,
    // an accent frame, and a bright value readout so it stands out in its section.
    LabelledFader (juce::AudioProcessorValueTreeState& apvts, const juce::String& pid,
        juce::String displayName, MidiLearnManager& learnMgr, bool emphasise = false)
        : LearnableComponent (learnMgr, pid), name (std::move (displayName)), emphasis (emphasise)
    {
        slider.setSliderStyle (juce::Slider::LinearVertical);
        slider.setTextBoxStyle (juce::Slider::NoTextBox, false, 0, 0);
        slider.setVelocityBasedMode (false); // absolute drag distance, not velocity
        slider.setSliderSnapsToMousePosition (false); // R2 GRAB mode: first touch acquires the
// control with NO value change; the value
// moves only on drag, relative to the grab
// point (snapping caused live-perf surprises).
        slider.setMouseDragSensitivity (kDragPixelsForFullRange); // R2: gentler drag-to-value
    }

```

```

    slider.setWantsKeyboardFocus (false);
    addAndMakeVisible (slider);
    attachment = std::make_unique<juce::SliderParameterAttachment> (*apvts.getParameter (pid), slider);

    param = apvts.getParameter (pid);
    enableNumericEntry (param);
    listenForLearnGestures (slider);
    setTooltipFromParam (apvts, &slider);      // hover -> full parameter name
}

// Custom hover help (overrides the param name); routes to the inner slider (the leaf).
void setHelp (const juce::String& text) { LearnableComponent::setHelp (text, &slider); }

// Transient readout: the value shows ONLY while the fader is being dragged (nothing at rest).
// Used where several narrow faders share a row (the ADSR envelope) so the values don't crush.
void setTransientReadout (bool on)
{
    transientReadout = on;
    slider.onDragStart = [this] { dragging = true; repaint(); };
    slider.onDragEnd   = [this] { dragging = false; repaint(); };
}

void paint (juce::Graphics& g) override
{
    if (emphasis)
    {
        // Distinct rounded frame so MASTER reads as the section's headline.
        auto r = getLocalBounds().toFloat().reduced (1.5f);
        g.setColour (VASynthLookAndFeel::accentWarm().withAlpha (0.10f));
        g.fillRoundedRectangle (r, 6.0f);
        g.setColour (VASynthLookAndFeel::accentWarm().withAlpha (0.75f));
        g.drawRoundedRectangle (r, 6.0f, 1.6f);
    }

    // Name: high-contrast (near-white / amber for master), larger for arm's-
    // length reading. Bold so it stays legible on a busy dark panel.
    g.setColour (emphasis ? VASynthLookAndFeel::accentWarm() : VASynthLookAndFeel::ink());
    g.setFont (juce::Font (juce::FontOptions (emphasis ? 15.0f : 13.0f, juce::Font::bold)));
    g.drawFittedText (emphasis ? name.toUpperCase() : name,
                     getLocalBounds().removeFromTop (labelH()), juce::Justification::centred, 1);

    // Live value readout with the parameter's own units/text (auto-fit width), in the accent
    // colour. When transient, it appears only while dragging (nothing at rest); time params
    // read in ms below 1 s, s above.
    if (! transientReadout || dragging)
    {
        g.setColour (emphasis ? VASynthLookAndFeel::ink() : VASynthLookAndFeel::accent());
        g.setFont (juce::Font (juce::FontOptions (juce::Font::getDefaultMonospacedFontName(),
                                                    emphasis ? 14.0f : 12.5f, juce::Font::bold)));
        g.drawFittedText (formatFaderValue(), getLocalBounds().removeFromBottom (labelH()),
                          juce::Justification::centred, 1);
    }

    paintLearnDecorations (g);
    paintModRing (g);
}

void resized() override
{
    const int pad = emphasis ? 6 : 2;
    slider.setBounds (getLocalBounds().reduced (pad, 0).withTrimmedTop (labelH() + 2).withTrimmedBottom
(labelH() + 2));
    if (ghost) { ghost->setBounds (slider.getBounds()); ghost->toFront (false); }
}

bool hasModIndicator() const override { return ghost != nullptr; }

protected:
    void modTargetAttached() override
    {
        if (ghost == nullptr) { ghost = std::make_unique<FaderModOverlay> (slider, *this); addAndMakeVisible
(*ghost); }
        ghost->begin();
        resized();
    }
}

```

```

private:
    int labelH() const { return emphasis ? 20 : 18; }

    // Time params (label "s") read as ms below 1 s, s above; everything else uses the param text.
    juce::String formatFaderValue() const
    {
        if (param != nullptr && param->getLabel() == "s")
        {
            const float sec = param->convertFrom0to1 (param->getValue());
            return sec < 1.0f ? juce::String (juce::roundToInt (sec * 1000.0f)) + " ms"
                : juce::String (sec, 2) + " s";
        }
        return formatParamValue (param);
    }

    bool transientReadout = false, dragging = false;

    // Mouse-transparent ghost thumb at the modulated position (same colour language as the knob
    // arc), distinct from the base thumb. Repaints ~30 Hz only while the route is actually moving.
    struct FaderModOverlay : juce::Component, private juce::Timer
    {
        FaderModOverlay (juce::Slider& sl, LearnableComponent& o) : s (sl), owner (o) { setInterceptsMouseClicks
(false, false); }
        void begin() { startTimerHz (modanim::kTimerHz); }
        void timerCallback() override
        {
            const int armed = owner.modArmedLfo();
            const bool moved = st.tick (owner.modAnim(), juce::Time::getMillisecondCounter());
            if (moved || armed != lastArmed) { lastArmed = armed; repaint(); }
        }
        void paint (juce::Graphics& g) override
        {
            const int armedLfo = owner.modArmedLfo(), lfoIdx = owner.modLfoIdx();
            const juce::Colour col = owner.modColour();
            auto b = getLocalBounds().toFloat();
            // #148 armed "editable picture": a static colour marker at the value marks membership.
            if (armedLfo >= 0 && lfoIdx >= 0)
            {
                const float y = b.getBottom() - (float) s.valueToProportionOfLength (s.getValue()) * b.getHeight();
                const bool mine = lfoIdx == armedLfo;
                g.setColour (VASynthLookAndFeel::lfoColour (lfoIdx).withAlpha (mine ? 0.95f : 0.28f));
                g.fillRoundedRectangle (b.getX() - 3.0f, y - (mine ? 2.5f : 1.5f), b.getWidth() + 6.0f, mine ? 5.0f
: 3.0f, 2.0f);
            }
            if (! st.visible()) return; // motion-gated
            float a = st.alpha();
            if (armedLfo >= 0 && lfoIdx != armedLfo) a *= 0.22f;
            const float base = (float) s.valueToProportionOfLength (s.getValue());
            const float curP = juce::jlimit (0.0f, 1.0f, base + st.cur);
            const float lagP = juce::jlimit (0.0f, 1.0f, base + st.lag);
            const float yCur = b.getBottom() - curP * b.getHeight(); // vertical fader: value up
            const float yLag = b.getBottom() - lagP * b.getHeight();
            g.setColour (col.withAlpha (0.35f * a));
            g.fillRect (b.getX() - 1.0f, juce::jmin (yCur, yLag), b.getWidth() + 2.0f, std::abs (yCur - yLag));
            g.setColour (col.withAlpha (0.9f * a));
            g.fillRoundedRectangle (b.getX() - 3.0f, yCur - 2.0f, b.getWidth() + 6.0f, 4.0f, 2.0f);
        }
    };
    juce::Slider& s; LearnableComponent& owner; modanim::State st; int lastArmed = -1;
};

juce::String name;
bool emphasis = false;
ModSlider slider { *this };
juce::RangedAudioParameter* param = nullptr;
std::unique_ptr<juce::SliderParameterAttachment> attachment;
std::unique_ptr<FaderModOverlay> ghost;

JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (LabelledFader)
};

// -----
// Horizontal fill-bar control: dragging/tapping left->right raises the value, and the FILLED BAR
// itself is the level indicator. Learnable + a registry mod-target + numeric-entry like the knobs
// and faders. Used for the NOISE source level.
class HBarControl : public LearnableComponent
{

```

```

public:
    HBarControl (juce::AudioProcessorValueTreeState& apvts, const juce::String& pid,
                 juce::String displayName, MidiLearnManager& learnMgr)
        : LearnableComponent (learnMgr, pid), name (std::move (displayName))
    {
        slider.setSliderStyle (juce::Slider::LinearBar);
        slider.setTextBoxStyle (juce::Slider::NoTextBox, false, 0, 0);
        slider.setSliderSnapsToMousePosition (true); // a fill bar: tap-to-position feels natural
        slider.setColour (juce::Slider::trackColourId, VASynthLookAndFeel::accent()); // the fill
        slider.setColour (juce::Slider::backgroundColourId, VASynthLookAndFeel::track()); // the trough
        slider.setWantsKeyboardFocus (false);
        addAndMakeVisible (slider);
        attachment = std::make_unique<juce::SliderParameterAttachment> (*apvts.getParameter (pid), slider);
        param = apvts.getParameter (pid);
        enableNumericEntry (param);
        listenForLearnGestures (slider);
        setTooltipFromParam (apvts, &slider);
    }

    void setHelp (const juce::String& text) { LearnableComponent::setHelp (text, &slider); }

    void paint (juce::Graphics& g) override
    {
        // The LinearBar slider paints the fill; overlay the value on the right for a readout.
        g.setColour (VASynthLookAndFeel::ink());
        g.setFont (juce::Font (juce::FontOptions (juce::Font::getDefaultMonospacedFontName(), 11.0f,
juce::Font::bold)));
        g.drawText (formatParamValue (param), getLocalBounds().reduced (7, 0), juce::Justification::centredRight,
false);
        paintLearnDecorations (g);
        paintModRing (g);
    }

    void resized() override
    {
        slider.setBounds (getLocalBounds());
        if (ghost) { ghost->setBounds (slider.getBounds()); ghost->toFront (false); }
    }

    bool hasModIndicator() const override { return ghost != nullptr; }

protected:
    // #12: a modulated NOISE level must animate like the knobs/faders. Without this override the
    // base no-op ran and no indicator was ever created (the control was wired but drew nothing).
    void modTargetAttached() override
    {
        if (ghost == nullptr) { ghost = std::make_unique<HBarModOverlay> (slider, *this); addAndMakeVisible
(*ghost); }
        ghost->begin();
        resized();
    }

private:
    // Mouse-transparent ghost marker at the modulated position (horizontal: value runs left->right).
    // Motion-gated + repaints ~30 Hz only while the route is actually moving (mirrors FaderModOverlay).
    struct HBarModOverlay : juce::Component, private juce::Timer
    {
        HBarModOverlay (juce::Slider& sl, LearnableComponent& o) : s (sl), owner (o) { setInterceptsMouseClicks
(false, false); }
        void begin() { startTimerHz (modanim::kTimerHz); }
        void timerCallback() override { if (st.tick (owner.modAnim(), juce::Time::getMillisecondCounter()))
repaint(); }
        void paint (juce::Graphics& g) override
        {
            if (! st.visible()) return; // motion-gated: nothing at rest
            const float a = st.alpha();
            const float base = (float) s.valueToProportionOfLength (s.getValue());
            const float curP = juce::jlimit (0.0f, 1.0f, base + st.cur);
            const float lagP = juce::jlimit (0.0f, 1.0f, base + st.lag);
            auto b = getLocalBounds().toFloat();
            const float xCur = b.getX() + curP * b.getWidth(); // horizontal bar: value right
            const float xLag = b.getX() + lagP * b.getWidth();
            g.setColour (VASynthLookAndFeel::accentWarm().withAlpha (0.35f * a)); // trail echo -> current
            g.fillRect (juce::jmin (xCur, xLag), b.getY() - 1.0f, std::abs (xCur - xLag), b.getHeight() + 2.0f);
            g.setColour (VASynthLookAndFeel::accentWarm().withAlpha (0.9f * a)); // ghost marker at current
            g.fillRoundedRectangle (xCur - 2.0f, b.getY() - 3.0f, 4.0f, b.getHeight() + 6.0f, 2.0f);
        }
    }

```

```

    }
    juce::Slider& s; LearnableComponent& owner; modanim::State st;
};

juce::String name;
ModSlider slider { *this };
juce::RangedAudioParameter* param = nullptr;
std::unique_ptr<juce::SliderParameterAttachment> attachment;
std::unique_ptr<HBarModOverlay> ghost;
JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (HBarControl)
};

// -----
// Transient toast notification (e.g. "Launchkey Mini connected"). Holds for ~2 s
// then fades. Intercepts no input and refuses focus, so it never disturbs the
// controls beneath it or the QWERTY note input.
class Toast : public juce::Component,
              private juce::Timer
{
public:
    Toast()
    {
        setInterceptsMouseClicks (false, false);
        setWantsKeyboardFocus (false);
        setVisible (false);
    }

    void show (const juce::String& message)
    {
        text = message;
        ticks = 0;
        alpha = 1.0f;
        setVisible (true);
        toFront (false);
        startTimerHz (30);
        repaint();
    }

    void paint (juce::Graphics& g) override
    {
        auto r = getLocalBounds().toFloat();
        g.setColour (VASynthLookAndFeel::panelLight().withAlpha (alpha * 0.96f));
        g.fillRoundedRectangle (r, 8.0f);
        g.setColour (VASynthLookAndFeel::accent().withAlpha (alpha));
        g.drawRoundedRectangle (r.reduced (0.75f), 8.0f, 1.4f);
        g.setColour (VASynthLookAndFeel::ink().withAlpha (alpha));
        g.setFont (juce::Font (juce::FontOptions (15.0f, juce::Font::bold)));
        g.drawFittedText (text, getLocalBounds().reduced (14, 0), juce::Justification::centred, 1);
    }

private:
    void timerCallback() override
    {
        ++ticks;
        if (ticks > 60) // ~2 s hold, then ~0.5 s fade
            alpha = juce::jmax (0.0f, 1.0f - (float) (ticks - 60) / 15.0f);
        if (alpha <= 0.0f) { setVisible (false); stopTimer(); }
        repaint();
    }

    juce::String text;
    float alpha = 1.0f;
    int ticks = 0;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (Toast)
};

// -----
// Rotary knob + name + live value, bound to a float parameter. MIDI-learnable and
// focus-refusing like the faders; used in the FX panel where knobs read better
// than a wall of faders.
class RotaryKnob : public LearnableComponent
{
public:
    // sideLabel: knob on the LEFT (square) with the name + value stacked to its
    // right — for wide/short rows (e.g. the filter's vertical knob column) where a

```

```

// name-above/value-below stack would leave the row half-empty.
RotaryKnob (juce::AudioProcessorValueTreeState& apvts, const juce::String& pid,
            juce::String displayName, MidiLearnManager& learnMgr, bool sideLabelLayout = false)
    : LearnableComponent (learnMgr, pid), name (std::move (displayName)), sideLabel (sideLabelLayout)
{
    slider.setSliderStyle (juce::Slider::RotaryVerticalDrag);
    slider.setTextBoxStyle (juce::Slider::NoTextBox, false, 0, 0);
    slider.setVelocityBasedMode (false); // relative drag distance (grab mode)
    slider.setSliderSnapsToMousePosition (false); // R2: acquire on touch, no value jump
    slider.setMouseDragSensitivity (kDragPixelsForFullRange); // R2: gentler drag-to-value
    slider.setWantsKeyboardFocus (false);
    addAndMakeVisible (slider);
    attachment = std::make_unique<juce::SliderParameterAttachment> (*apvts.getParameter (pid), slider);

    param = apvts.getParameter (pid);
    enableNumericEntry (param);
    listenForLearnGestures (slider);
    setTooltipFromParam (apvts, &slider); // hover -> full parameter name
}

// Custom hover help (overrides the param name); routes to the inner slider (the leaf).
void setHelp (const juce::String& text) { LearnableComponent::setHelp (text, &slider); }

void paint (juce::Graphics& g) override
{
    const juce::String text = formatParamValue (param);
    if (sideLabel)
    {
        auto lab = getLocalBounds().withTrimmedLeft (getHeight() + 4);
        g.setColour (VASynthLookAndFeel::ink());
        g.setFont (juce::Font (juce::FontOptions (13.0f, juce::Font::bold)));
        g.drawFittedText (name, lab.removeFromTop (lab.getHeight() * 3 / 5), juce::Justification::centredLeft,
1);
        g.setColour (VASynthLookAndFeel::accent());
        g.setFont (juce::Font (juce::FontOptions (juce::Font::getDefaultMonospacedFontName(), 11.5f,
juce::Font::bold)));
        g.drawFittedText (text, lab, juce::Justification::centredLeft, 1);
        paintLearnDecorations (g);
        paintModRing (g);
        return;
    }

    // Name: shrink-to-fit (down to 0.55x) across up to nameLines lines, so a long macro
    // assignment ("Resonance", "Filter Env Amt") reads in full rather than ellipsizing.
    g.setColour (VASynthLookAndFeel::ink());
    g.setFont (juce::Font (juce::FontOptions (nameLines > 1 ? 11.5f : 12.5f, juce::Font::bold)));
    g.drawFittedText (name, getLocalBounds().removeFromTop (nameTopH()),
        juce::Justification::centredTop, nameLines, 0.55f);

    // Value: always for showValue knobs; for a transient readout, only while dragging (nothing
    // at rest) – restores the macros' readout without the clutter.
    if (showValue || (transientReadout && dragging))
    {
        g.setColour (VASynthLookAndFeel::accent());
        g.setFont (juce::Font (juce::FontOptions (juce::Font::getDefaultMonospacedFontName(), 12.0f,
juce::Font::bold)));
        g.drawFittedText (text, getLocalBounds().removeFromBottom (15), juce::Justification::centred, 1);
    }

    paintLearnDecorations (g);
    paintModRing (g);
}

void resized() override
{
    if (sideLabel)
        slider.setBounds (getLocalBounds().removeFromLeft (getHeight()));
    else
        slider.setBounds (getLocalBounds().withTrimmedTop (nameTopH() + 1).withTrimmedBottom (showValue ? 16 :
2));
    if (overlay) { overlay->setBounds (slider.getBounds()); overlay->toFront (false); }
}

// Allow the name to wrap to N lines (macros use 2 so full assignment names fit). The CC chip
// moves to the bottom so it never overlaps the taller name.
void setNameLines (int n) { nameLines = juce::jmax (1, n); ccBadgeAtBottom = nameLines > 1; resized();

```

```

repaint(); }

// Transient value readout: the value shows only while the knob is being dragged (nothing at rest).
void setTransientReadout (bool on)
{
    transientReadout = on;
    slider.onDragStart = [this] { dragging = true; repaint(); };
    slider.onDragEnd   = [this] { dragging = false; repaint(); };
}

// Update the displayed name (e.g. a macro showing its assigned target).
void setDisplayName (juce::String n) { if (n != name) { name = std::move (n); repaint(); } }

// Override drag sensitivity for this knob (pixels of travel for the full range;
// fewer px = more responsive). Default is kDragPixelsForFullRange for all controls.
void setDragPixels (int px) { slider.setMouseDragSensitivity (juce::jmax (1, px)); }
int dragPixels() const { return slider.getMouseDragSensitivity(); } // for tests / audits

// Accept BOTH horizontal and vertical drag (default is vertical only). For knobs high in
// the window (the top-bar macros) this lets a touch drag sideways to adjust instead of
// being forced upward into the OS title bar, which on a windowed touch screen would grab
// the drag and move the window. The mouse cursor is pinned during any rotary drag anyway.
void setBothAxisDrag() { slider.setSliderStyle (juce::Slider::RotaryHorizontalVerticalDrag); }

// Hide the numeric value readout (macros – keeps the top bar compact).
void setShowValue (bool b) { if (b != showValue) { showValue = b; resized(); repaint(); } }

bool hasModIndicator() const override { return overlay != nullptr; }

protected:
// The base wired this control as a mod target: bring up the arc animation reading modAnim().
void modTargetAttached() override
{
    if (overlay == nullptr) { overlay = std::make_unique<ModOverlay> (slider); addAndMakeVisible (*overlay); }
    overlay->modFn      = [this] { return modAnim(); };
    overlay->colourFn   = [this] { return modColour(); }; // arc tinted by the driving LFO
    overlay->lfoIdxFn   = [this] { return modLfoIdx(); }; // for the per-LFO dash (colour-blind cue)
    overlay->armedFn    = [this] { return modArmedLfo(); }; // arm-mode "editable picture"
    overlay->boundsFn   = [this] { return inBoundsCapture() ? juce::Point<float> (boundsLo(), boundsHi()) :
juce::Point<float> (-1.0f, -1.0f); };
    overlay->begin();
    resized();
}

private:
int nameTopH() const { return nameLines > 1 ? 24 : 16; }
int nameLines = 1;
bool transientReadout = false, dragging = false;

// Moving tick + faint span showing the live LFO-modulated position on the knob arc.
// Mouse-transparent (never blocks dragging); geometry mirrors VASynthLookAndFeel's rotary.
struct ModOverlay : juce::Component, private juce::Timer
{
    explicit ModOverlay (juce::Slider& sl) : s (sl) { setInterceptsMouseClicks (false, false); }
    void begin() { startTimerHz (modanim::kTimerHz); }
    void timerCallback() override
    {
        const int armed = armedFn ? armedFn() : -1; // arm-mode changes must repaint even at rest
        const bool moved = modFn && st.tick (modFn(), juce::Time::getMillisecondCounter());
        if (moved || armed != lastArmed) { lastArmed = armed; repaint(); }
    }
    // Per-LFO colour-blind cue: LF01 solid, LF02 dashed, LF03 dotted.
    void strokeCued (juce::Graphics& g, const juce::Path& p, float w, juce::Colour c, int lfoIdx)
    {
        g.setColour (c);
        juce::PathStrokeType st2 (w, juce::PathStrokeType::curved, juce::PathStrokeType::rounded);
        if (lfoIdx == 1 || lfoIdx == 2)
        {
            const float dash1[] { w * 2.4f, w * 1.6f }; // LF02: dashed
            const float dash2[] { w * 0.9f, w * 1.6f }; // LF03: dotted
            juce::Path dp; st2.createDashedStroke (dp, p, lfoIdx == 1 ? dash1 : dash2, 2);
            g.fillPath (dp);
        }
        else g.strokePath (p, st2);
    }
    void paint (juce::Graphics& g) override

```

```

{
    const int armedLfo = armedFn ? armedFn() : -1; // LFO armed for link (else -1)
    const int lfoIdx = lfoIdxFn ? lfoIdxFn() : -1; // LFO colouring this dest (else -1)
    const juce::Colour col = colourFn ? colourFn() : VASynthLookAndFeel::accentWarm();
    auto b = getLocalBounds().toFloat().reduced(3.0f);
    const float radius = juce::jmin(b.getWidth(), b.getHeight()) * 0.5f;
    const float cx = b.getCentreX(), cy = b.getCentreY();
    const float lineW = juce::jmax(2.5f, radius * 0.16f);
    const float arcR = radius - lineW * 0.5f;

    // Arm mode = "editable picture": a STATIC ring marks this dest's link membership.
    if (armedLfo >= 0 && lfoIdx >= 0)
    {
        juce::Path ring; ring.addCentredArc(cx, cy, arcR, arcR, 0.0f, 0.0f,
juce::MathConstants<float>::twoPi, true);
        const bool mine = lfoIdx == armedLfo; // linked to the armed LFO -> bold; other LFO ->
faint
        strokeCued(g, ring, mine ? lineW : lineW * 0.7f,
VASynthLookAndFeel::lfoColour(lfoIdx).withAlpha(mine ? 0.95f : 0.28f), lfoIdx);
    }

    // Inc 3 live bounds readout: while sliding to set bounds, draw the [lo,hi] arc + the %,
    // since the knob rests at the midpoint after release (the drag is otherwise invisible).
    const juce::Point<float> bnd = boundsFn ? boundsFn() : juce::Point<float>(-1.0f, -1.0f);
    if (bnd.x >= 0.0f)
    {
        const auto rp2 = s.getRotaryParameters();
        const auto a2 = [&](float p) { return rp2.startAngleRadians + p * (rp2.endAngleRadians -
rp2.startAngleRadians); };
        const juce::Colour bc = (armedLfo >= 0 ? VASynthLookAndFeel::lfoColour(armedLfo) : col);
        juce::Path span; span.addCentredArc(cx, cy, arcR, arcR, 0.0f, a2(bnd.x), a2(bnd.y), true);
        g.setColour(bc.withAlpha(0.9f));
        g.strokePath(span, juce::PathStrokeType(lineW * 1.2f, juce::PathStrokeType::curved,
juce::PathStrokeType::rounded));
        g.setFont(juce::Font(juce::FontOptions(10.0f, juce::Font::bold)));
        g.drawText(juce::String(juce::roundToInt(bnd.x * 100)) + "-" + juce::String(juce::roundToInt
(bnd.y * 100)) + "%",
getLocalBounds(), juce::Justification::centred, false);
        return; // during the drag, show only the bounds
    }

    if (! modFn || ! st.visible()) return; // motion-gated animated arc
    float a = st.alpha();
    if (armedLfo >= 0 && lfoIdx != armedLfo) a *= 0.22f; // dim non-armed-LFO motion while arming
    const auto rp = s.getRotaryParameters();
    const float base = (float) s.valueToProportionOfLength(s.getValue());
    const float cur = juce::jlimit(0.0f, 1.0f, base + st.cur);
    const float lag = juce::jlimit(0.0f, 1.0f, base + st.lag);
    const auto ang = [&](float p) { return rp.startAngleRadians + p * (rp.endAngleRadians -
rp.startAngleRadians); };
    juce::Path trail; trail.addCentredArc(cx, cy, arcR, arcR, 0.0f, ang(lag), ang(cur), true);
    strokeCued(g, trail, lineW, col.withAlpha(0.45f * a), lfoIdx);
    const float a1 = ang(cur);
    const juce::Point<float> pt(cx + std::sin(a1) * arcR, cy - std::cos(a1) * arcR);
    g.setColour(col.withAlpha(a));
    g.fillEllipse(pt.x - 3.0f, pt.y - 3.0f, 6.0f, 6.0f);
}
juce::Slider& s;
std::function<float>() modFn;
std::function<juce::Colour>() colourFn;
std::function<int>() lfoIdxFn, armedFn;
std::function<juce::Point<float>()> boundsFn; // (lo,hi) while sliding to bounds, else (-1,-1)
int lastArmed = -1;
modanim::State st;
};

juce::String name;
bool sideLabel = false;
bool showValue = true;
ModSlider slider { *this }; // the shared LINK-aware slider (see LearnableComponent)

juce::RangedAudioParameter* param = nullptr;
std::unique_ptr<juce::SliderParameterAttachment> attachment;
std::unique_ptr<ModOverlay> overlay;

JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(RotaryKnob)

```

```

};

// -----
// Touch-friendly segmented button row for a choice parameter (one tap, visible
// state) – preferred over a dropdown where the option count allows.
class SegmentedControl : public LearnableComponent
{
public:
    SegmentedControl (juce::AudioProcessorValueTreeState& apvts, const juce::String& pid,
                     juce::String displayName, MidiLearnManager& learnMgr)
        : LearnableComponent (learnMgr, pid), name (std::move (displayName))
    {
        choice = dynamic_cast<juce::AudioParameterChoice*> (apvts.getParameter (pid));
        jassert (choice != nullptr);

        // Segmented controls need more width than a single fader so labels read.
        getProperties().set ("layoutFlex", juce::jmax (2.2, 0.85 * choice->choices.size()));

        for (int i = 0; i < choice->choices.size(); ++i)
        {
            auto* b = buttons.add (new juce::TextButton (choice->choices[i]));
            b->setClickingTogglesState (false);
            b->setWantsKeyboardFocus (false);
            b->setColour (juce::TextButton::buttonColourId, VASynthLookAndFeel::track());
            b->setColour (juce::TextButton::buttonOnColourId, VASynthLookAndFeel::accent());
            b->setColour (juce::TextButton::textColourOffId, VASynthLookAndFeel::dim());
            b->setColour (juce::TextButton::textColourOnId, juce::Colours::black);
            const int idx = i;
            b->onClick = [this, idx] { setIndex (idx); };
            b->setTooltip (choice->getName (128)); // hover -> full parameter name
            addAndMakeVisible (b);
            listenForLearnGestures (*b);
        }
        setTooltipFromParam (apvts);
        attachment = std::make_unique<juce::ParameterAttachment> (
            *choice, [this] (float) { refresh(); }, nullptr);
        attachment->sendInitialUpdate();
    }

    void paint (juce::Graphics& g) override
    {
        g.setColour (VASynthLookAndFeel::ink());
        g.setFont (juce::Font (juce::FontOptions (13.0f, juce::Font::bold)));
        g.drawText (name, getLocalBounds().removeFromTop (18), juce::Justification::centred, false);
        paintLearnDecorations (g);
    }

    void resized() override
    {
        // Stack buttons vertically – full-width finger targets filling the column.
        auto r = getLocalBounds().withTrimmedTop (18);
        const int n = buttons.size();
        if (n == 0) return;
        const int h = r.getHeight() / n;
        for (int i = 0; i < n; ++i)
            buttons[i]->setBounds (r.removeFromTop (i == n - 1 ? r.getHeight() : h).reduced (2, 2));
    }

private:
    void setIndex (int i)
    {
        choice->beginChangeGesture();
        choice->setValueNotifyingHost (choice->convertTo0to1 ((float) i));
        choice->endChangeGesture();
        refresh();
    }

    void refresh()
    {
        const int current = choice->getIndex();
        for (int i = 0; i < buttons.size(); ++i)
            buttons[i]->setToggleState (i == current, juce::dontSendNotification);
    }

    juce::String name;
    juce::AudioParameterChoice* choice = nullptr;
    juce::OwnedArray<juce::TextButton> buttons;

```

```

std::unique_ptr<juce::ParameterAttachment> attachment;

JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (SegmentedControl)
};

// -----
// Horizontal one-tap selector for a choice parameter – headerless (the section
// header names it), buttons spread left-to-right. This is the R2 layout's segmented
// grammar (osc wave, LFO dest, filter type, chord degree/quality). Optional short
// labels override the parameter's long choice names ("Square" -> "SQR"). Optionally
// draws its own tinted labels above/beside (headerless by default). MIDI-learnable
// and focus-refusing like every control.
class HSelector : public LearnableComponent
{
public:
    HSelector (juce::AudioProcessorValueTreeState& apvts, const juce::String& pid,
               MidiLearnManager& learnMgr, juce::StringArray labelOverride = {})
        : LearnableComponent (learnMgr, pid)
    {
        choice = dynamic_cast<juce::AudioParameterChoice*> (apvts.getParameter (pid));
        jassert (choice != nullptr);
        labels = labelOverride.isEmpty() ? choice->choices : labelOverride;

        for (int i = 0; i < choice->choices.size(); ++i)
        {
            auto* b = buttons.add (new juce::TextButton (labels[juce::jmin (i, labels.size() - 1)]));
            b->setClickingTogglesState (false);
            b->setWantsKeyboardFocus (false);
            b->setColour (juce::TextButton::buttonColourId, VASynthLookAndFeel::track());
            b->setColour (juce::TextButton::buttonOnColourId, VASynthLookAndFeel::accent());
            b->setColour (juce::TextButton::textColourOffId, VASynthLookAndFeel::ink());
            b->setColour (juce::TextButton::textColourOnId, juce::Colours::black);
            const int idx = i;
            // Tapping the button that is ALREADY current fires onReselect (no value change) –
            // used by the WT wave button to open the table picker on a second tap. A tap on any
            // other button is a normal selection.
            b->onClick = [this, idx] { if (choice->getIndex() == idx && onReselect) onReselect (idx); else setIndex
(id); };
            b->setTooltip (choice->getName (128)); // hover -> full parameter name
            addAndMakeVisible (b);
            listenForLearnGestures (*b);
        }
        setTooltipFromParam (apvts);
        attachment = std::make_unique<juce::ParameterAttachment> (
            *choice, [this] (float) { refresh(); }, nullptr);
        attachment->sendInitialUpdate();
    }

    void paint (juce::Graphics& g) override { paintLearnDecorations (g); }

    // Fired when the currently-selected button is tapped again (no value change). Optional;
    // the WT wave button uses it to open the table picker on a second tap.
    std::function<void (int)> onReselect;

    void resized() override
    {
        auto r = getLocalBounds();
        const int n = buttons.size();
        if (n == 0) return;
        for (int i = 0; i < n; ++i)
        {
            auto cell = juce::Rectangle<int> (r.getX() + i * r.getWidth() / n, r.getY(),
                                               r.getWidth() / n, r.getHeight());
            buttons[i]->setBounds (cell.reduced (2));
        }
    }

private:
    void setIndex (int i)
    {
        choice->beginChangeGesture();
        choice->setValueNotifyingHost (choice->convertTo0to1 ((float) i));
        choice->endChangeGesture();
        refresh();
    }
    void refresh()

```

```

{
    const int current = choice->getIndex();
    for (int i = 0; i < buttons.size(); ++i)
        buttons[i]->setToggleState (i == current, juce::dontSendNotification);
}

juce::AudioParameterChoice* choice = nullptr;
juce::StringArray labels;
juce::OwnedArray<juce::TextButton> buttons;
std::unique_ptr<juce::ParameterAttachment> attachment;

JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (HSelector)
};

// -----
// LFO shape picker drawn as four stacked waveform ICONS (Triangle / Sine / Square /
// S&H) bound to a choice parameter — the R2 LFO grammar. Tap an icon to select.
// MIDI-learnable + focus-refusing.
class ShapeSelector : public LearnableComponent
{
public:
    ShapeSelector (juce::AudioProcessorValueTreeState& apvts, const juce::String& pid,
                  MidiLearnManager& learnMgr)
        : LearnableComponent (learnMgr, pid)
    {
        choice = dynamic_cast<juce::AudioParameterChoice*> (apvts.getParameter (pid));
        jassert (choice != nullptr);
        setTooltipFromParam (apvts); // hover -> full parameter name
        attachment = std::make_unique<juce::ParameterAttachment> (
            *choice, [this] (float) { repaint(); }, nullptr);
        attachment->sendInitialUpdate();
    }

    // Draw one waveform icon (0 Tri, 1 Sin, 2 Sqr, 3 S&H) filling r.
    static void drawIcon (juce::Graphics& g, juce::Rectangle<int> r, int kind, bool on)
    {
        g.setColour (on ? VASynthLookAndFeel::accent() : VASynthLookAndFeel::track());
        g.fillRoundedRectangle (r.toFloat(), 4.0f);
        auto a = r.toFloat().reduced (r.getWidth() * 0.18f, r.getHeight() * 0.28f);
        const float x0 = a.getX(), w = a.getWidth(), y0 = a.getCentreY(), h = a.getHeight() * 0.5f;
        g.setColour (on ? juce::Colour (0xff0e1319) : VASynthLookAndFeel::ink());
        juce::Path p;
        if (kind == 0) { p.startNewSubPath (x0, y0); p.lineTo (x0+w*0.25f, y0-h); p.lineTo (x0+w*0.75f, y0+h);
        p.lineTo (x0+w, y0); }
        else if (kind == 1) { p.startNewSubPath (x0, y0); for (int i = 1; i <= 20; ++i) { float t = i/20.0f;
        p.lineTo (x0+w*t, y0 - std::sin (t*6.283f)*h); } }
        else if (kind == 2) { p.startNewSubPath (x0, y0+h); p.lineTo (x0, y0-h); p.lineTo (x0+w*0.5f, y0-h);
        p.lineTo (x0+w*0.5f, y0+h); p.lineTo (x0+w, y0+h); p.lineTo (x0+w, y0-h); }
        else { const float s[5] { 0.3f, -0.6f, 0.5f, -0.2f, 0.7f }; float px = x0; for (int i = 0; i < 5;
        ++i) { float ny = y0 - s[i]*h; p.startNewSubPath (px, ny); p.lineTo (px+w/5.0f, ny); px += w/5.0f; } }
        g.strokePath (p, juce::PathStrokeType (1.6f));
    }

    void paint (juce::Graphics& g) override
    {
        const int cur = choice != nullptr ? choice->getIndex() : 0;
        const int n = 4;
        const int ih = (getHeight() - (n - 1) * gap) / n;
        for (int k = 0; k < n; ++k)
        {
            cells[(std::size_t) k] = juce::Rectangle<int> (0, k * (ih + gap), getWidth(), ih);
            drawIcon (g, cells[(std::size_t) k], k, k == cur);
        }
        paintLearnDecorations (g);
    }

    void mouseUp (const juce::MouseEvent& e) override
    {
        LearnableComponent::mouseUp (e);
        if (e.mods.isPopupMenu() || e.getDistanceFromDragStart() > 8) return;
        for (int k = 0; k < 4; ++k)
            if (cells[(std::size_t) k].contains (e.getPosition()) && choice != nullptr)
            {
                choice->beginChangeGesture();
                choice->setValueNotifyingHost (choice->convertTo0to1 ((float) k));
                choice->endChangeGesture();
            }
    }
};

```

```

        repaint();
        return;
    }

private:
    static constexpr int gap = 3;
    juce::AudioParameterChoice* choice = nullptr;
    std::array<juce::Rectangle<int>, 4> cells {};
    std::unique_ptr<juce::ParameterAttachment> attachment;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (ShapeSelector)
};

===== FILE 75/75: Source/VersionInfo.h.in (11 lines) =====
// Synth – Copyright © 2026 John L Farmer. Licensed under AGPLv3; see LICENSE.
#pragma once
// GENERATED each build by cmake/gen_version.cmake – always the currently-built commit
// ("+" suffix = uncommitted changes present). Do not edit; edit VersionInfo.h.in instead.
#define VASYNTH_GIT_HASH_RT "@GITHASH@"

// Uniquely-greppable build stamp so tooling (run-all-checks.sh) can prove a shipped binary was
// built from the gated commit – grep the artefact for "VASYNTHBUILD:<hash>". The literal prefix
// makes the match collision-proof vs. a bare 7-hex token appearing in the binary's byte noise.
// A "+" suffix (same as the banner) flags a build from an uncommitted tree.
#define VASYNTH_BUILD_STAMP "VASYNTHBUILD:@GITHASH@"

```